

```

1  import { _decorator, assetManager, CCInteger, director, Director, Enum, Node, NodeEventType, SpriteAtlas,
2  SpriteFrame, UITransform, UIOpacity, UIRenderer, Vec2 } from 'cc';
3  import { EDITOR, JSB } from 'cc/env';
4  enum SizeMode { CUSTOM, TRIMMED, RAW }
5  const { ccclass, property } = _decorator;
6  @ccclass('Corner')
7  class Corner {
8      @property({ displayName: '↙ 左下' })
9      leftBottom: boolean = true;
10     @property({ displayName: '↘ 右下' })
11     rightBottom: boolean = true;
12     @property({ displayName: '↖ 左上' })
13     leftTop: boolean = true;
14     @property({ displayName: '↗ 右上' })
15     rightTop: boolean = true;
16     visible: boolean[] = null;
17 }
18 @ccclass('RoundBox')
19 export class RoundBox extends UIRenderer {
20     @property({ displayName: '图集', type: SpriteAtlas, readonly: true, editorOnly: true, serializable: false })
21     private atlas: SpriteAtlas = null;
22     @property
23     private _spriteFrame: SpriteFrame = null;
24     @property({ displayName: 'Sprite Frame', type: SpriteFrame })
25     private get spriteFrame() { return this._spriteFrame; }
26     private set spriteFrame(val) {
27         this._spriteFrame = val;
28         this.updateSpriteFrame();
29         this.updateUv();
30         if (this._renderData?.chunk) {
31             this.markForUpdateRenderData();
32         }
33     }
34     /**
35      * 运行时换图（皮肤等），与编辑器「Sprite Frame」一致。
36      * 节点未启用或非激活子节点上时尚无 _renderData，只写入 _spriteFrame，等 onEnable 里会
37     updateUv。
38      */
39     public setDisplaySpriteFrame(val: SpriteFrame | null): void {
40         this._spriteFrame = val;
41         this.updateSpriteFrame();
42         if (this._renderData?.chunk) {
43             this.updateUv();
44             this.markForUpdateRenderData();
45         }
46     }
47     @property
48     private _sizeMode: SizeMode = SizeMode.TRIMMED;
49     @property({ displayName: '尺寸模式', type: Enum(SizeMode) })
50     private get sizeMode() { return this._sizeMode; }
51     private set sizeMode(val) {
52         this._sizeMode = val;
53         this.updateSizeMode();
54     }
55     @property({ displayName: '顶点数 / 三角形数', readonly: true, editorOnly: true, serializable: false })
56     private vertexTriangle: Vec2 = new Vec2(0, 0);
57     @property
58     private _segment: number = 5;

```

```

59     @property({ displayName: '..... 线段数量', type: CCInteger })
60     private get segment() { return this._segment; }
61     private set segment(val) {
62         this._segment = Math.max(val, 1);
63         this.createData();
64         this.updateLocal();
65         this.updateUv();
66         this.updateColor();
67         this.markForUpdateRenderData();
68     }
69     @property
70     private _radius: number = 100;
71     @property({ displayName: '..... 圆角半径' })
72     private get radius() { return this._radius; }
73     private set radius(val) {
74         this._radius = Math.max(val, 0);
75         this.updateLocal();
76         this.updateUv();
77         this.markForUpdateRenderData();
78     }
79     @property
80     private _corner: Corner = new Corner();
81     @property({ displayName: '..... 圆角可见性' })
82     private get corner() { return this._corner; }
83     private set corner(val) {
84         this._corner = val;
85         this.updateCorner();
86         this.createData();
87         this.updateLocal();
88         this.updateUv();
89         this.updateColor();
90         this.markForUpdateRenderData();
91     }
92     private uiTrans: UITransform = null;    //当前节点的 UITransform 对象
93     private left: number = 0;              //左边缘本地坐标
94     private bottom: number = 0;            //下边缘本地坐标
95     private locals: number[][] = [];       //顶点本地坐标
96     /** 上一帧链式 UIOpacity 乘积; 父节点 tween 时引擎不会每帧调 updateColor, 需自行对齐顶点
97     alpha */
98     private _lastCascadeOpacityMul = -1;
99     /**
100     * `MenuOverlayWindow` 等下一帧 `forceRefreshRender` 时, 可能早于本组件 `__preload`, `uiTrans` 尚
101     未赋值。
102     */
103     private ensureUiTransform(): boolean {
104         if (!this.node?.isValid) {
105             return false;
106         }
107         if (!this.uiTrans || !this.uiTrans.isValid) {
108             this.uiTrans = this.node.getComponent(UITransform) ?? this.node.addComponent(UITransform);
109         }
110         return !(this.uiTrans && this.uiTrans.isValid);
111     }
112     __preload(): void {
113         super.__preload();
114         this._assembler = {                //定制 assembler
115             updateColor: this.updateColor.bind(this),
116             updateRenderData: this.updateRenderData.bind(this),

```

```

117         fillBuffers: this.fillBuffer.bind(this),
118     };
119     this.uiTrans = this.node.getComponent(UITransform) ?? this.node.addComponent(UITransform);
120     this._useVertexOpacity = true;
121     this['updateMaterial']();
122     this.updateSpriteFrame();
123     this.updateCorner();
124     this.updateLocal();
125 }
126 onEnable(): void {
127     super.onEnable();
128     this._lastCascadeOpacityMul = -1;
129     this.createData();
130     this.updateUv();
131     JSB ? director.once(Director.EVENT_AFTER_DRAW, this.updateColor, this) : this.updateColor();
132     this.node.on(NodeEventType.SIZE_CHANGED, this.onSizeChanged, this);
133     this.node.on(NodeEventType.ANCHOR_CHANGED, this.onAnchorChanged, this);
134 }
135 /**
136  * 整棵 UI 曾处于 inactive、首次被打开时，部分环境下首帧 `_spriteFrame.texture` /
137  * 尚未就绪，`updateUv` 会空跑，圆角底图不显示；下一帧再算一次即可（由 MenuOverlayWindow 等
138  * 调用）。
139  */
140
141 forceRefreshRender(): void {
142     if (!this.isValid || !this.ensureUiTransform()) {
143         return;
144     }
145     /** MenuOverlayWindow 等下一帧回调时，可能早于 __preload 里对 corner.visible 的赋值 */
146     this.updateCorner();
147     this.createData();
148     this.updateLocal();
149     this.updateUv();
150     this.updateXy();
151     this.updateColor();
152     this.markForUpdateRenderData();
153 }
154 onDisable(): void {
155     super.onDisable();
156     this.node.off(NodeEventType.SIZE_CHANGED, this.onSizeChanged, this);
157     this.node.off(NodeEventType.ANCHOR_CHANGED, this.onAnchorChanged, this);
158 }
159 //修改节点尺寸后，更新顶点数据，并根据 sizeMode 设置图片宽高
160 private onSizeChanged(): void {
161     if (!this._spriteFrame) return;
162     if (!this.ensureUiTransform()) return;
163     this.updateLocal();
164     this.updateXy();
165     let cw = this.uiTrans.width, ch = this.uiTrans.height;
166     let size = this._spriteFrame['_originalSize'], rect = this._spriteFrame['_rect'];
167     switch (this._sizeMode) {
168         case SizeMode.TRIMMED: if (cw === rect.width && ch === rect.height) return; break;
169         case SizeMode.RAW: if (cw === size.width && ch === size.height) return; break;
170     }
171     this._sizeMode = SizeMode.CUSTOM;
172 }
173 private onAnchorChanged(): void {
174     this.updateLocal();

```

```

175         this.updateXy();
176     }
177     //可以传入单张图片 Texture2D, 或 Atlas 图集帧 (支持合批, 推荐)
178     private updateSpriteFrame(): void {
179         let spriteFrame = this._spriteFrame;
180         if (!spriteFrame) { this.atlas = null; return; }
181         this._renderData && (this._renderData.textureDirty = true);
182         this.updateSizeMode();
183         if (EDITOR) {
184             if (!spriteFrame['_atlasUuid']) { this.atlas = null; return; }
185             assetManager.loadAny(spriteFrame['_atlasUuid'], (err: Error, asset: SpriteAtlas) => {
186                 if (err) { this.atlas = null; return; }
187                 this.atlas = asset;
188             });
189         }
190     }
191     private updateCorner(): void {
192         if (!this._corner) {
193             this._corner = new Corner();
194         }
195         const corner = this._corner;
196         corner.visible = [corner.leftBottom, corner.rightBottom, corner.rightTop, corner.leftTop];
197     }
198     //设置顶点数和三角形数
199     private createData(): void {
200         if (!this._corner?.visible || this._corner.visible.length < 4) {
201             this.updateCorner();
202         }
203         const renderData = (this._renderData = this.requestRenderData());
204         if (!renderData) {
205             return;
206         }
207         let vertexTriangle = [4, 2];
208         let cornerCnt = 0;
209         const visible = this._corner.visible;
210         for (let i = 0; i < 4; visible[i++] && ++cornerCnt);
211         vertexTriangle = [12 + (this._segment - 1) * cornerCnt, 6 + this._segment * cornerCnt];
212         renderData.dataLength = vertexTriangle[0];
213         renderData.resize(vertexTriangle[0], 3 * vertexTriangle[1]);
214         EDITOR && ([this.vertexTriangle.x, this.vertexTriangle.y] = vertexTriangle);
215         this.updateIndices();
216     }
217     //计算顶点索引
218     private updateIndices(): void {
219         const renderData = this._renderData;
220         if (!renderData?.chunk) {
221             return;
222         }
223         let indices = new Uint16Array(renderData.chunk.indexCount);
224         const ROUND_IB = [0, 9, 11, 0, 11, 1, 2, 8, 10, 2, 4, 8, 3, 5, 7, 3, 7, 6];
225         for (let i = ROUND_IB.length - 1; i > -1; indices[i] = ROUND_IB[i--]);
226         for (let i = 0, offset = ROUND_IB.length, id = 36, visible = this._corner.visible; i < 4; ++i) {
227             if (!visible[i]) continue;
228             let o = 3 * i;
229             let a = o + 1;
230             let b = id / 3;
231             for (let j = 0, len = this._segment - 1; j < len; ++j) {
232                 indices[offset++] = o;

```

```

233             indices[offset++] = a;
234             indices[offset++] = b;
235             a = b++;
236             id += 3;
237         }
238         indices[offset++] = o;
239         indices[offset++] = a;
240         indices[offset++] = o + 2;
241     }
242     JSB ? renderData.chunk.setIndexBuffer(indices) : (renderData.indices = indices);
243 }
244 //计算本地坐标
245 updateLocal(): void {
246     if (!this.ensureUiTransform()) {
247         return;
248     }
249     if (!this._corner?.visible || this._corner.visible.length < 4) {
250         this.updateCorner();
251     }
252     let ut = this.uiTrans, cw = ut.width, ch = ut.height, ax = ut.anchorX, ay = ut.anchorY;
253     let l = this.left = -cw * ax, b = this.bottom = -ch * ay, r = cw * (1 - ax), t = ch * (1 - ay);
254     let locals = this.locals = [];
255     let radius = Math.min(this._radius, Math.min(cw, ch) / 2);
256     let lo = l + radius, bo = b + radius, ro = r - radius, to = t - radius;
257     let corner = this._corner;
258     locals[0] = [lo, corner.leftBottom ? bo : b];
259     locals[1] = [l, locals[0][1]];
260     locals[2] = [lo, b];
261     locals[3] = [ro, corner.rightBottom ? bo : b];
262     locals[4] = [ro, b];
263     locals[5] = [r, locals[3][1]];
264     locals[6] = [ro, corner.rightTop ? to : t];
265     locals[7] = [r, locals[6][1]];
266     locals[8] = [ro, t];
267     locals[9] = [lo, corner.leftTop ? to : t];
268     locals[10] = [lo, t];
269     locals[11] = [l, locals[9][1]];
270     let radian = Math.PI / (this._segment << 1);
271     let sin = Math.sin(radian), cos = Math.cos(radian);
272     for (let i = 0, offset = 12, visible = corner.visible; i < 4; ++i) {
273         if (!visible[i]) continue;
274         let id = i * 3;
275         let ox = locals[id][0], oy = locals[id][1];
276         let delTX = locals[id + 1][0] - ox, delTY = locals[id + 1][1] - oy;
277         for (let j = 0, len = this._segment - 1; j < len; ++j) {
278             locals[offset] = [ox + delTX * cos - delTY * sin, oy + delTY * cos + delTX * sin];
279             delTX = locals[offset][0] - ox;
280             delTY = locals[offset][1] - oy;
281             ++offset;
282         }
283     }
284 }
285 //根据本地坐标，计算 XY 坐标
286 updateXy(): void {
287     let renderData = this._renderData, data = renderData.data, locals = this.locals;
288     for (let i = locals.length - 1; i > -1; --i) {
289         let local = this.locals[i];
290         data[i].x = local[0];

```

```

291         data[i].y = local[1];
292     }
293     !JSB && (renderData.vertDirty = true);
294 }
295 updateUv(): void {
296     let spriteFrame = this._spriteFrame;
297     if (!spriteFrame) return;
298     if (!this.ensureUITransform()) return;
299     let renderData = this._renderData;
300     if (!renderData?.chunk) return;
301     let vb = renderData.chunk.vb, locals = this.locals;
302     let ut = this.uiTrans, cw = ut.width, ch = ut.height, l = this.left, b = this.bottom;
303     for (let i = 3, len = vb.length, step = renderData.floatStride, id = 0; i < len; i += step, ++id) {
304         let local = locals[id];
305         vb[i] = (local[0] - l) / cw;
306         vb[i + 1] = (local[1] - b) / ch;
307     }
308     let uv = spriteFrame.uv;
309     if (spriteFrame['_rotated']) {
310         let uvL = uv[0], uvB = uv[1], uvW = uv[4] - uvL, uvH = uv[3] - uvB;
311         for (let i = 3, len = vb.length, step = renderData.floatStride; i < len; i += step) {
312             let tmp = vb[i];
313             vb[i] = uvL + vb[i + 1] * uvW;
314             vb[i + 1] = uvB + tmp * uvH;
315         }
316     } else {
317         let uvL = uv[0], uvB = uv[1], uvW = uv[2] - uvL, uvH = uv[5] - uvB;
318         for (let i = 3, len = vb.length, step = renderData.floatStride; i < len; i += step) {
319             vb[i] = uvL + vb[i] * uvW;
320             vb[i + 1] = uvB + vb[i + 1] * uvH;
321         }
322     }
323 }
324 /**
325  * VERTEX 填充色类型下, Batch2D 不会对顶点再乘 updateOpacity (见引擎 _handleUIRenderer 里
326  * VERTEX 分支)。
327  * 因此必须把节点链上 UIOpacity 的乘积并入顶点 alpha, 否则父节点 tween UIOpacity (如 Toast 根)
328  * 时底图不会变淡。
329  */
330 private cascadeUiOpacityMultiplier(): number {
331     let m = 1;
332     for (let n: Node | null = this.node; n; n = n.parent) {
333         const ui = n.getComponent(UIOpacity);
334         if (ui && ui.enabledInHierarchy) {
335             m *= ui.opacity / 255;
336         }
337     }
338     return m;
339 }
340 //计算顶点颜色
341 updateColor(): void {
342     let renderData = this._renderData;
343     if (!renderData?.chunk) return;
344     let vb = renderData.chunk.vb, color = this._color;
345     const opacityMul = this.cascadeUiOpacityMultiplier();
346     this._lastCascadeOpacityMul = opacityMul;
347     let r = color.r / 255, g = color.g / 255, b = color.b / 255, a = (color.a / 255) * opacityMul;
348     for (let i = 5, len = vb.length, step = renderData.floatStride; i < len; i += step) {

```

```

349         vb[i] = r;
350         vb[i + 1] = g;
351         vb[i + 2] = b;
352         vb[i + 3] = a;
353     }
354     renderData.chunk.meshBuffer?.setDirty();
355 }
356 protected lateUpdate(): void {
357     if (!this._renderData?.chunk || !this._spriteFrame?.texture) {
358         return;
359     }
360     const m = this.cascadeUiOpacityMultiplier();
361     if (Math.abs(m - this._lastCascadeOpacityMul) < 1 / 2550) {
362         return;
363     }
364     this.updateColor();
365     this.markForUpdateRenderData();
366 }
367 //根据尺寸模式，修改节点尺寸
368 updateSizeMode(): void {
369     if (!this._spriteFrame) return;
370     if (!this.ensureUiTransform()) return;
371     switch (this._sizeMode) {
372         case SizeMode.TRIMMED: this.uiTrans.setContentSize(this._spriteFrame['_rect'].size); break;
373         case SizeMode.RAW: this.uiTrans.setContentSize(this._spriteFrame['_originalSize']); break;
374     }
375 }
376 //调用 markForUpdateRenderData 后该函数会被触发
377 updateRenderData(): void {
378     if (!this._renderData || !this._spriteFrame?.texture) return;
379     this.updateXy();
380     this._renderData.updateRenderData(this, this._spriteFrame);
381 }
382 protected _render(render: any): void {
383     render.commitComp(this, this._renderData, this._spriteFrame, this._assembler, null);
384 }
385 protected _canRender(): boolean {
386     return super._canRender() && !(this._spriteFrame?.texture);
387 }
388 //Web 平台，将 renderData 的数据提交给 GPU 渲染
389 //原生平台并不会执行该函数，引擎另外实现了渲染函数
390 fillBuffer(): void {
391     let renderData = this._renderData;
392     if (!renderData) return;
393     let chunk = renderData.chunk;
394     if (this.node.hasChangedFlags || renderData.vertDirty) {
395         let data = renderData.data, vb = renderData.chunk.vb, m = this.node.worldMatrix;
396         for (let step = renderData.floatStride, len = vb.length, i = 0, id = 0; i < len; i += step, ++id) {
397             let x = data[id].x, y = data[id].y, rhw = m.m03 * x + m.m07 * y + m.m15;
398             rhw = rhw ? 1 / rhw : 1;
399             vb[i] = (m.m00 * x + m.m04 * y + m.m12) * rhw;
400             vb[i + 1] = (m.m01 * x + m.m05 * y + m.m13) * rhw;
401         }
402         renderData.vertDirty = false;
403     }
404     let vid = chunk.vertexOffset;
405     let meshBuffer = chunk.meshBuffer;
406     let iData = meshBuffer.iData;

```

```

407         let indexOffset = meshBuffer.indexOffset;
408         let indices = renderData.indices;
409         for (let i = 0; i < indices.length; iData[indexOffset++] = vid + indices[i++]);
410         meshBuffer.indexOffset += indices.length;
411     }
412 }
413 declare global {
414     module gi {
415         class RoundBox extends UIRenderer {
416             static SizeMode: typeof SizeMode;           //尺寸模式枚举
417             spriteFrame: SpriteFrame;                   //纹理，支持单张图片和 Atlas 图集帧
418             sizeMode: SizeMode;                         //尺寸模式
419             roundSegment: number;                       //线段数量
420             roundRadius: number;                       //圆角半径
421             roundCorner: Corner;                       //圆角可见性
422         }
423     }
424 }
425 ((globalThis as any).gi || {}).RoundBox ||= Object.assign(RoundBox, { SizeMode: SizeMode });
426 // ----- assets/Scripts/Component/BrownianSquareFieldView.ts -----
427 import {
428     _decorator,
429     Color,
430     Component,
431     Graphics,
432     Node,
433     UITransform,
434 } from 'cc';
435 import { clamp } from '../Utils/Utils';
436 const { ccclass } = _decorator;
437 /**#region 参数（只改这里；勿用 @property，避免场景序列化旧值覆盖脚本）
438 /** 小正方形数量 n */
439 const SQUARE_COUNT = 25;
440 /** 边长基准 x（设计像素） */
441 const SIDE_BASE = 60;
442 /** 边长随机幅度 y：实际边长  $\in [x-y, x+y]$  */
443 const SIDE_JITTER = 20;
444 /** 边长目标值随机游走（设计像素/秒），在  $[x-y, x+y]$  内缩放 */
445 const SIDE_SCALE_TARGET_WANDER = 10;
446 /** 边长向目标靠拢速度（越大缩放响应越快） */
447 const SIDE_SCALE_LERP = 4;
448 /** 边框厚度占边长比例 a（0.08 = 8%） */
449 const BORDER_THICKNESS_RATIO = 0;
450 /** 圆角半径占边长比例 b（0.12 = 12%） */
451 const CORNER_RADIUS_RATIO = 0.1;
452 /** 边框不透明度基准 d（0~255） */
453 const OPACITY_BASE = 10;
454 /** 不透明度随机幅度 c（0.2 =  $\pm 20\%$  相对 d） */
455 const OPACITY_JITTER_RATIO = 0.2;
456 /** 边框相对填充的 RGB 深度（0.55 = 边框更暗；1 则与填充同色） */
457 const STROKE_DARKEN_RATIO = 0.55;
458 /** 游走速度基准 v（设计像素/秒） */
459 const SPEED_BASE = 20;
460 /** 速度随机幅度 u：实际速度  $\in [v-u, v+u]$  */
461 const SPEED_JITTER = 5;
462 /** 布朗运动：每秒速度方向最大抖动（弧度） */
463 const BROWNIAN_ANGLE_JITTER = 2.8;
464 /** 布朗运动：每秒速度标量随机游走强度 */

```

```

465 const BROWNIAN_SPEED_WANDER = 28;
466 /** 初始撒点：在网格单元内随机偏移幅度（0.7 = 单元 70% 范围内抖动） */
467 const SPAWN_CELL_JITTER = 0.7;
468 /** 运行时互斥强度（越大越均匀；0 关闭） */
469 const SEPARATION_STRENGTH = 1;
470 /** 互斥迭代次数（每帧，2~3 通常够用） */
471 const SEPARATION_ITERATIONS = 2;
472 /** 中心最小间距 = (两边长之和)/2 × 该系数（<1 允许轻贴，>1 留缝） */
473 const SEPARATION_GAP_RATIO = 1.4;
474 /** 子节点容器名 */
475 const PARTICLE_LAYER_NAME = 'BrownianSquareLayer';
476 //endregion
477 interface IBrownianSquareParticle {
478     node: Node;
479     graphics: Graphics;
480     side: number;
481     /** 边长缩放目标（布朗游走，限制在  $x \pm y$ ） */
482     sideTarget: number;
483     opacity: number;
484     borderRatio: number;
485     cornerRatio: number;
486     speed: number;
487     angle: number;
488 }
489 /**
490  * 挂载在任意带 UITransform 的节点上：按节点尺寸生成 n 个圆角小正方形（填充与边框同色），
491  * 在范围内布朗运动游走。改参数请编辑本文件顶部常量区。
492  */
493 @ccclass('BrownianSquareFieldView')
494 export class BrownianSquareFieldView extends Component {
495     private _layerNode: Node | null = null;
496     private _particles: IBrownianSquareParticle[] = [];
497     private _boundsLeft = 0;
498     private _boundsRight = 0;
499     private _boundsBottom = 0;
500     private _boundsTop = 0;
501     private _lastFieldWidth = -1;
502     private _lastFieldHeight = -1;
503     protected onLoad(): void {
504         this._ensureFieldTransform();
505         this._rebuildField();
506     }
507     protected onEnable(): void {
508         this._rebuildField();
509     }
510     protected onDisable(): void {
511         this._clearParticles();
512     }
513     protected update(dt: number): void {
514         if (this._particles.length === 0) {
515             return;
516         }
517         if (this._syncBoundsIfResized()) {
518             this._rebuildField();
519             return;
520         }
521         this._stepBrownianMotion(dt);
522     }

```

```
523  /** 尺寸变更后手动重建 */
524  rebuild(): void {
525      this._rebuildField();
526  }
527  //region 构建
528  private _ensureFieldTransform(): UITransform | null {
529      const ut = this.getComponent(UITransform) ?? this.addComponent(UITransform);
530      return ut;
531  }
532  private _ensureLayerNode(): Node {
533      let layer = this.node.getChildByName(PARTICLE_LAYER_NAME);
534      if (!layer?.isValid) {
535          layer = new Node(PARTICLE_LAYER_NAME);
536          layer.setParent(this.node);
537      }
538      layer.setPosition(0, 0, 0);
539      if (!layer.getComponent(UITransform)) {
540          layer.addComponent(UITransform);
541      }
542      this._layerNode = layer;
543      return layer;
544  }
545  private _syncBoundsIfResized(): boolean {
546      const ut = this._ensureFieldTransform();
547      if (!ut) {
548          return false;
549      }
550      const w = ut.width;
551      const h = ut.height;
552      if (w === this._lastFieldWidth && h === this._lastFieldHeight) {
553          return false;
554      }
555      return true;
556  }
557  private _updateBounds(): void {
558      const ut = this._ensureFieldTransform();
559      if (!ut) {
560          return;
561      }
562      const w = ut.width;
563      const h = ut.height;
564      this._lastFieldWidth = w;
565      this._lastFieldHeight = h;
566      this._boundsLeft = -ut.anchorX * w;
567      this._boundsRight = this._boundsLeft + w;
568      this._boundsBottom = -ut.anchorY * h;
569      this._boundsTop = this._boundsBottom + h;
570  }
571  private _rebuildField(): void {
572      this._updateBounds();
573      this._clearParticles();
574      const layer = this._ensureLayerNode();
575      const count = Math.max(0, Math.floor(SQUARE_COUNT));
576      for (let i = 0; i < count; i++) {
577          this._particles.push(this._createParticle(layer, i, count));
578      }
579      if (SEPARATION_STRENGTH > 0) {
580          this._applySpatialSeparation(SEPARATION_ITERATIONS * 3);
```

```
581     }
582   }
583   private _createParticle(layer: Node, index: number, total: number): IBrownianSquareParticle {
584     const side = this._randomSide();
585     const opacity = this._randomOpacity();
586     const speed = this._randomSpeed();
587     const angle = Math.random() * Math.PI * 2;
588     const half = side * 0.5;
589     const { x, y } = this._sampleGridSpawnPosition(index, total, half);
590     const node = new Node(`BrownianSq_${index}`);
591     node.setParent(layer);
592     node.setPosition(x, y, 0);
593     const ut = node.addComponent(UITransform);
594     ut.setContentSize(side, side);
595     ut.setAnchorPoint(0.5, 0.5);
596     const graphics = node.addComponent(Graphics);
597     const particle: IBrownianSquareParticle = {
598       node,
599       graphics,
600       side,
601       sideTarget: side,
602       opacity,
603       borderRatio: BORDER_THICKNESS_RATIO,
604       cornerRatio: CORNER_RADIUS_RATIO,
605       speed,
606       angle,
607     };
608     this._drawSquareStroke(particle);
609     return particle;
610   }
611   private _drawSquareStroke(p: IBrownianSquareParticle): void {
612     const g = p.graphics;
613     if (!g?.isValid) {
614       return;
615     }
616     const side = p.side;
617     const half = side * 0.5;
618     const lineWidth = Math.max(1, side * p.borderRatio);
619     const radius = clamp(side * p.cornerRatio, 0, half);
620     g.clear();
621     g.lineWidth = lineWidth;
622     g.lineJoin = Graphics.LineJoin.ROUND;
623     g.lineCap = Graphics.LineCap.ROUND;
624     const alpha = clamp(Math.round(p.opacity), 0, 255);
625     const strokeRgb = clamp(Math.round(255 * STROKE_DARKEN_RATIO), 0, 255);
626     g.fillColor = new Color(255, 255, 255, alpha);
627     g.strokeColor = new Color(strokeRgb, strokeRgb, strokeRgb, alpha);
628     g.roundRect(-half, -half, side, side, radius);
629     g.fill();
630     g.stroke();
631   }
632   private _clearParticles(): void {
633     for (const p of this._particles) {
634       if (p.node?.isValid) {
635         p.node.destroy();
636       }
637     }
638     this._particles.length = 0;
```

```

639     }
640     //endregion
641     //region 空间平衡
642     /** 按网格分区生成初始位置，避免纯随机留下大面积空白 */
643     private _sampleGridSpawnPosition(index: number, total: number, half: number): { x: number; y: number } {
644         const minX = this._boundsLeft + half;
645         const maxX = this._boundsRight - half;
646         const minY = this._boundsBottom + half;
647         const maxY = this._boundsTop - half;
648         const spanX = maxX - minX;
649         const spanY = maxY - minY;
650         const centerX = (minX + maxX) * 0.5;
651         const centerY = (minY + maxY) * 0.5;
652         if (spanX <= 0 || spanY <= 0 || total <= 0) {
653             return { x: centerX, y: centerY };
654         }
655         const aspect = spanX / Math.max(spanY, 1);
656         const cols = Math.max(1, Math.ceil(Math.sqrt(total * aspect)));
657         const rows = Math.max(1, Math.ceil(total / cols));
658         const col = index % cols;
659         const row = Math.floor(index / cols);
660         const cellW = spanX / cols;
661         const cellH = spanY / rows;
662         const jitter = clamp(SPAWN_CELL_JITTER, 0, 1);
663         const x = clamp(
664             minX + (col + 0.5) * cellW + (Math.random() - 0.5) * cellW * jitter,
665             minX,
666             maxX,
667         );
668         const y = clamp(
669             minY + (row + 0.5) * cellH + (Math.random() - 0.5) * cellH * jitter,
670             minY,
671             maxY,
672         );
673         return { x, y };
674     }
675     /** 运行时轻量互斥：过近则推开，减轻聚堆与大块空白 */
676     private _applySpatialSeparation(iterations: number): void {
677         if (SEPARATION_STRENGTH <= 0 || this._particles.length < 2) {
678             return;
679         }
680         const strength = clamp(SEPARATION_STRENGTH, 0, 1);
681         const gapRatio = Math.max(0.1, SEPARATION_GAP_RATIO);
682         for (let pass = 0; pass < iterations; pass++) {
683             for (let i = 0; i < this._particles.length; i++) {
684                 const a = this._particles[i];
685                 if (!a.node?.isValid) {
686                     continue;
687                 }
688                 const posA = a.node.position;
689                 let ax = posA.x;
690                 let ay = posA.y;
691                 for (let j = i + 1; j < this._particles.length; j++) {
692                     const b = this._particles[j];
693                     if (!b.node?.isValid) {
694                         continue;
695                     }
696                     const posB = b.node.position;

```

```

697         let bx = posB.x;
698         let by = posB.y;
699         const dx = bx - ax;
700         const dy = by - ay;
701         const distSq = dx * dx + dy * dy;
702         const minDist = (a.side + b.side) * 0.5 * gapRatio;
703         if (minDist <= 0) {
704             continue;
705         }
706         if (distSq >= minDist * minDist) {
707             continue;
708         }
709         let dist = Math.sqrt(distSq);
710         if (dist < 1e-4) {
711             const angle = Math.random() * Math.PI * 2;
712             const push = minDist * 0.5 * strength;
713             ax -= Math.cos(angle) * push;
714             ay -= Math.sin(angle) * push;
715             bx += Math.cos(angle) * push;
716             by += Math.sin(angle) * push;
717         } else {
718             const overlap = (minDist - dist) * 0.5 * strength;
719             const nx = dx / dist;
720             const ny = dy / dist;
721             ax -= nx * overlap;
722             ay -= ny * overlap;
723             bx += nx * overlap;
724             by += ny * overlap;
725         }
726         const aHalf = a.side * 0.5;
727         const bHalf = b.side * 0.5;
728         ax = clamp(ax, this._boundsLeft + aHalf, this._boundsRight - aHalf);
729         ay = clamp(ay, this._boundsBottom + aHalf, this._boundsTop - aHalf);
730         bx = clamp(bx, this._boundsLeft + bHalf, this._boundsRight - bHalf);
731         by = clamp(by, this._boundsBottom + bHalf, this._boundsTop - bHalf);
732         a.node.setPosition(ax, ay, 0);
733         b.node.setPosition(bx, by, 0);
734     }
735 }
736 }
737 }
738 private _clampParticleToBounds(p: IBrownianSquareParticle, x: number, y: number): { x: number; y: number }
739 {
740     const half = p.side * 0.5;
741     return {
742         x: clamp(x, this._boundsLeft + half, this._boundsRight - half),
743         y: clamp(y, this._boundsBottom + half, this._boundsTop - half),
744     };
745 }
746 //endregion
747 //region 布朗运动
748 private _stepBrownianMotion(dt: number): void {
749     const safeDt = clamp(dt, 0, 0.05);
750     for (const p of this._particles) {
751         if (!p.node?.isValid) {
752             continue;
753         }
754         this._stepSideScale(p, safeDt);

```

```

755         p.angle += (Math.random() - 0.5) * 2 * BROWNIAN_ANGLE_JITTER * safeDt;
756         p.speed = clamp(
757             p.speed + (Math.random() - 0.5) * 2 * BROWNIAN_SPEED_WANDER * safeDt,
758             this._speedMin(),
759             this._speedMax(),
760         );
761         let vx = Math.cos(p.angle) * p.speed;
762         let vy = Math.sin(p.angle) * p.speed;
763         const pos = p.node.position;
764         let x = pos.x + vx * safeDt;
765         let y = pos.y + vy * safeDt;
766         const half = p.side * 0.5;
767         const minX = this._boundsLeft + half;
768         const maxX = this._boundsRight - half;
769         const minY = this._boundsBottom + half;
770         const maxY = this._boundsTop - half;
771         if (x < minX) {
772             x = minX;
773             vx = Math.abs(vx);
774         } else if (x > maxX) {
775             x = maxX;
776             vx = -Math.abs(vx);
777         }
778         if (y < minY) {
779             y = minY;
780             vy = Math.abs(vy);
781         } else if (y > maxY) {
782             y = maxY;
783             vy = -Math.abs(vy);
784         }
785         p.angle = Math.atan2(vy, vx);
786         const clamped = this._clampParticleToBounds(p, x, y);
787         p.node.setPosition(clamped.x, clamped.y, 0);
788     }
789     this._applySpatialSeparation(SEPARATION_ITERATIONS);
790 }
791 //endregion
792 //region 随机参数
793 private _sideMin(): number {
794     return Math.max(4, SIDE_BASE - Math.max(0, SIDE_JITTER));
795 }
796 private _sideMax(): number {
797     return SIDE_BASE + Math.max(0, SIDE_JITTER);
798 }
799 private _randomSide(): number {
800     const min = this._sideMin();
801     const max = this._sideMax();
802     return min + Math.random() * (max - min);
803 }
804 /** 边长在 x±y 范围内随机缩放, 并同步重绘与边界 */
805 private _stepSideScale(p: IBrownianSquareParticle, dt: number): void {
806     const minSide = this._sideMin();
807     const maxSide = this._sideMax();
808     p.sideTarget += (Math.random() - 0.5) * 2 * SIDE_SCALE_TARGET_WANDER * dt;
809     p.sideTarget = clamp(p.sideTarget, minSide, maxSide);
810     const prevSide = p.side;
811     const lerpT = clamp(SIDE_SCALE_LERP * dt, 0, 1);
812     p.side = prevSide + (p.sideTarget - prevSide) * lerpT;

```

```

813         if (Math.abs(p.side - prevSide) < 0.2) {
814             return;
815         }
816         const ut = p.node.getComponent(UITransform);
817         if (ut) {
818             ut.setContentSize(p.side, p.side);
819         }
820         this._drawSquareStroke(p);
821         const pos = p.node.position;
822         const clamped = this._clampParticleToBounds(p, pos.x, pos.y);
823         if (clamped.x !== pos.x || clamped.y !== pos.y) {
824             p.node.setPosition(clamped.x, clamped.y, 0);
825         }
826     }
827     private _randomOpacity(): number {
828         const ratio = Math.max(0, OPACITY_JITTER_RATIO);
829         const factor = 1 + (Math.random() * 2 - 1) * ratio;
830         return clamp(OPACITY_BASE * factor, 0, 255);
831     }
832     private _randomSpeed(): number {
833         const jitter = Math.max(0, SPEED_JITTER);
834         return clamp(SPEED_BASE + (Math.random() * 2 - 1) * jitter, 0, this._speedMax());
835     }
836     private _speedMin(): number {
837         return Math.max(0, SPEED_BASE - Math.max(0, SPEED_JITTER));
838     }
839     private _speedMax(): number {
840         return Math.max(this._speedMin(), SPEED_BASE + Math.max(0, SPEED_JITTER));
841     }
842     //endregion
843 }
844 // ----- assets/Scripts/Config/DebugMockSave.ts -----
845 /**
846  * true: 仅在进程内第一次 DataManager.init 时把 MOCK_PLAYER_DATA 写入本地再 restore（避免切场景
847 第二次 init 覆盖你在 Menu 的修改）。
848  * false: 完全读本地已有存档，不写 mock。
849  * 发版前请改为 false。
850  */
851 export const USE_DEBUG MOCK_SAVE = true;
852 /** 可只写关心的字段；其余在 restore 时与默认存档合并。仅当 USE_DEBUG MOCK_SAVE 为 true 时写
853 入本地 */
854 export const MOCK_PLAYER_DATA: {
855     /** 菜单「开始游戏」进入的关卡 / 上次退出关卡（与解锁无关，可任意已解锁关） */
856     opticalCurrentLevelId?: number;
857     /** 已解锁 / 通关记录: [levelId, steps, ...]; steps=999999 表示仅解锁未通；解锁以出现过的 levelId
858 为准（支持跳关） */
859     opticalLevelClears?: number[];
860     bgmOn?: boolean;
861     sfxOn?: boolean;
862 } = {
863     /** 继续游戏目标关（须在 clears 已解锁的 id 中） */
864     opticalCurrentLevelId: 35,
865     /** 1/2/3/5 已通；4/6 未在表中 → 选关锁定；无 999999 占位也可，有真实步数即视为已解锁 */
866     opticalLevelClears: [1, 999999, 2, 999999, 3, 999999, 4, 999999, 5, 999999, 6, 999999, 7, 999999, 8, 999999,
867 9, 999999, 10, 999999, 11, 999999, 12, 999999, 13, 999999, 14, 999999, 15, 999999, 16, 999999, 17, 999999, 18,
868 999999, 19, 999999, 20, 999999, 21, 999999, 22, 999999, 23, 999999, 24, 999999, 25, 999999, 26, 999999, 27,
869 999999, 28, 999999, 29, 999999, 30, 999999, 31, 999999, 32, 999999, 33, 999999, 34, 999999, 35, 999999, 36,
870 999999, 37, 999999, 38, 999999, 39, 999999, 40, 999999, 41, 999999, 42, 999999, 43, 999999, 44, 999999, 45,

```

```

871 999999, 46, 999999, 47, 999999, 48, 999999, 49, 999999, 50, 999999, 51, 999999, 52, 999999, 53, 999999, 54,
872 999999, 55, 999999, 56, 999999, 57, 999999, 58, 999999, 59, 999999, 60, 999999, 61, 999999, 62, 999999, 63,
873 999999, 64, 999999, 65, 999999, 66, 999999, 67, 999999, 68, 999999, 69, 999999, 70, 999999, 71, 999999, 72,
874 999999, 73, 999999, 74, 999999, 75, 999999, 76, 999999, 77, 999999, 78, 999999, 79, 999999, 80, 999999, 81,
875 999999, 82, 999999, 83, 999999, 84, 999999, 85, 999999, 86, 999999, 87, 999999, 88, 999999, 89, 999999, 90,
876 999999, 91, 999999, 92, 999999, 93, 999999, 94, 999999, 95, 999999, 96, 999999, 97, 999999, 98, 999999, 99,
877 999999, 100, 999999],
878 };
879 // ----- assets/Scripts/Games/OpticalPuzzle/Application/OpticalPuzzleSession.ts -----
880 import { DEV_LEVEL_MINIMAL, type IOpticalLevelConfig } from '../Config/OpticalPuzzleLevelSchema';
881 import { OpticalPuzzleCore, type OpticalPlayStateSnapshot } from '../Core/OpticalPuzzleCore';
882 import type { OpticalBeamSnapshot } from '../Core/OpticalPuzzleCore';
883 import { Direction, MoveAttemptResult, type OpticalBoardSnapshot } from '../Core/OpticalPuzzleTypes';
884 /** OPTICAL_SNAPSHOT_CHANGED 载荷: 棋盘快照 + 本局移动步数 + 触发原因 */
885 export interface OpticalSnapshotNotify {
886     snapshot: OpticalBoardSnapshot;
887     moveCount: number;
888     notifyReason?: OpticalSessionNotifyReason;
889 }
890 import { AUDIO_EFFECT_ENUM, EVENT_ENUM } from '../Utils/Enum';
891 import { PLAY_AUDIO, SHOW_TOAST } from '../Utils/Event';
892 import { OpticalGameFlowState } from '../OpticalPuzzleStateMachine';
893 const UNDO_STEPS = 1;
894 /** 撤回键图标横向耗尽阶段: 0 满 → 3 空 (每按一次 +1, 各减 33% 右缘填充) */
895 export const UNDO_ICON_FILL_STAGES = 3;
896 export type OpticalSessionNotifyReason = 'load' | 'move' | 'face' | 'push' | 'undo' | 'reset' | 'complete';
897 /** 已在初始状态时撤回/重开的 Toast 参数 */
898 const INITIAL_STATE_TOAST = {
899     message: '已在初始状态',
900     bgWidth: 360,
901     localY: -250,
902 } as const;
903 /**
904  * 关卡会话: 加载配置、维护历史栈、撤回、重置。
905  * 不依赖场景节点。
906  */
907 export class OpticalPuzzleSession {
908     readonly core = new OpticalPuzzleCore();
909     private _level: IOpticalLevelConfig | null = null;
910     private _history: OpticalPlayStateSnapshot[] = [];
911     private _flow: OpticalGameFlowState = OpticalGameFlowState.BOOT;
912     /** 撤回键图标填充阶段 0~UNDO_ICON_FILL_STAGES */
913     private _undoFillStage = 0;
914     /** 本局有效移动步数 (成功移动/推箱 +1, 撤回 -1, 重开/开局 0) */
915     private _moveCount = 0;
916     /** 本局已观看激励视频解锁参考解的关卡 id (换关清零; 重开本关不清) */
917     private _answerUnlockedLevelId = -1;
918     /** Presentation 订阅: 参数用于区分移动 / 撤回 / 重置等 (音效与埋点) */
919     onStateChanged: ((reason: OpticalSessionNotifyReason) => void) | null = null;
920     get flowState(): OpticalGameFlowState {
921         return this._flow;
922     }
923     /** 撤回键图标填充阶段 (0 全填, 3 不填) */
924     get undoFillStage(): number {
925         return this._undoFillStage;
926     }
927     /** 本局已消耗移动步数 */
928     get moveCount(): number {

```

```

929         return this._moveCount;
930     }
931     /** 是否存在可撤回的有效操作（玩家或元件相对上一检查点有变化） */
932     canUndo(): boolean {
933         return this._flow === OpticalGameFlowState.RUNNING && this._history.length > 1;
934     }
935     /** 局内进行中且相对开局无任何操作记录 */
936     isAtInitialPlayState(): boolean {
937         return this._flow === OpticalGameFlowState.RUNNING && this._history.length <= 1;
938     }
939     private _emitInitialStateToast(): void {
940         SHOW_TOAST.emit(EVENT_ENUM.SHOW_TOAST, { ...INITIAL_STATE_TOAST });
941     }
942     /** 撤回键按下：图标右缘按 33% 递进清空（仅在实际撤回成功时调用） */
943     registerUndoButtonPress(): void {
944         if (this._undoFillStage < UNDO_ICON_FILL_STAGES) {
945             this._undoFillStage += 1;
946         }
947     }
948     /** 图标填充已耗尽（stage=3），再按撤回应走激励广告 */
949     isUndoIconFillExhausted(): boolean {
950         return this._undoFillStage >= UNDO_ICON_FILL_STAGES;
951     }
952     /** 激励广告完整观看后：恢复撤回图标满填，可再按三次 */
953     restoreUndoFillFromRewardedAd(): void {
954         this._resetUndoFillStage();
955     }
956     private _resetUndoFillStage(): void {
957         this._undoFillStage = 0;
958     }
959     /** 使用内置开发关启动（可改为 loadLevel(cfg)） */
960     startWithDevLevel(): void {
961         this.loadLevel(DEV_LEVEL_MINIMAL);
962     }
963     loadLevel(level: IOpticalLevelConfig): void {
964         const prevLevelId = this._level?.levelId ?? -1;
965         this._level = level;
966         if (prevLevelId !== level.levelId) {
967             this._answerUnlockedLevelId = -1;
968         }
969         this.core.reset(level);
970         this._history.length = 0;
971         this._resetUndoFillStage();
972         this._moveCount = 0;
973         this._pushHistory();
974         this._flow = OpticalGameFlowState.RUNNING;
975         this._emit('load');
976     }
977     getSnapshot(): OpticalBoardSnapshot {
978         return this.core.getSnapshot();
979     }
980     getBeamSnapshot(): OpticalBeamSnapshot {
981         return this.core.getBeamSnapshot();
982     }
983     /** 本关参考解是否已在本局通过激励视频解锁 */
984     isAnswerUnlocked(): boolean {
985         return this._level != null && this._answerUnlockedLevelId === this._level.levelId;
986     }

```

```

987      /** 激励视频完整观看后解锁本关参考解（仅本局有效） */
988      unlockAnswerForCurrentLevel(): void {
989          this._answerUnlockedLevelId = this._level?.levelId ?? -1;
990      }
991      /** 四向按钮 / 键盘入口；非 RUNNING 时返回 null */
992      applyDirection(dir: Direction): MoveAttemptResult | null {
993          if (this._flow !== OpticalGameFlowState.RUNNING) {
994              return null;
995          }
996          this.core.setPlayerFacing(dir);
997          const prevTargetLit = this.core.getSnapshot().targets.map((t) => t.lit);
998          const r = this.core.tryMove(dir);
999          if (r === MoveAttemptResult.Blocked) {
1000              // 与 BoardView 「><」 阻拦表情（reason=face）同一条件：凡 Blocked 即失败反馈
1001              PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.OPTICAL_MOVE_FAIL);
1002              this._emit('face');
1003              return r;
1004          }
1005          if (r === MoveAttemptResult.PlayerMoved || r === MoveAttemptResult.PiecePushed) {
1006              this._moveCount += 1;
1007              this._pushHistory();
1008              this._emitMoveSuccessSounds(prevTargetLit);
1009              if (this.core.isAllTargetsLit()) {
1010                  this._flow = OpticalGameFlowState.SETTLEMENT;
1011                  this._emit('complete');
1012              } else if (r === MoveAttemptResult.PiecePushed) {
1013                  this._emit('push');
1014              } else {
1015                  this._emit('move');
1016              }
1017          }
1018          return r;
1019      }
1020      /** 移动成功音；若本步有新亮灯则与点亮音同帧叠播（无先后顺序） */
1021      private _emitMoveSuccessSounds(prevLit: boolean[]): void {
1022          const sfx: AUDIO_EFFECT_ENUM[] = [AUDIO_EFFECT_ENUM.OPTICAL_MOVE_SUCCESS];
1023          if (this._hasNewlyLitTarget(prevLit)) {
1024              sfx.push(AUDIO_EFFECT_ENUM.OPTICAL_TARGET_LIT);
1025          }
1026          PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, sfx);
1027      }
1028      private _hasNewlyLitTarget(prevLit: boolean[]): boolean {
1029          const targets = this.core.getSnapshot().targets;
1030          for (let i = 0; i < targets.length; i++) {
1031              if (targets[i].lit && !prevLit[i]) {
1032                  return true;
1033              }
1034          }
1035          return false;
1036      }
1037      /** 撤回：撤销 UNDO_STEPS 步有效操作（当前为 1 步）；无可撤回步时弹 Toast 并返回 false */
1038      undoBatch(): boolean {
1039          if (!this.canUndo()) {
1040              if (this.isAtInitialPlayState()) {
1041                  this._emitInitialStateToast();
1042              }
1043              return false;
1044          }

```

```

1045         const pops = Math.min(UNDO_STEPS, this._history.length - 1);
1046         for (let i = 0; i < pops; i++) {
1047             this._history.pop();
1048         }
1049         const snap = this._history[this._history.length - 1];
1050         this.core.restorePlayState(snap);
1051         this._moveCount = Math.max(0, this._moveCount - pops);
1052         this._emit('undo');
1053         return true;
1054     }
1055     resetLevel(): void {
1056         if (!this._level) {
1057             return;
1058         }
1059         if (this.isAtInitialPlayState()) {
1060             this._emitInitialStateToast();
1061             return;
1062         }
1063         this.core.reset(this._level);
1064         this._history.length = 0;
1065         this._resetUndoFillStage();
1066         this._moveCount = 0;
1067         this._pushHistory();
1068         this._flow = OpticalGameFlowState.RUNNING;
1069         this._emit('reset');
1070     }
1071     private _pushHistory(): void {
1072         this._history.push(this.core.clonePlayState());
1073     }
1074     private _emit(reason: OpticalSessionNotifyReason): void {
1075         this.onStateChanged?.(reason);
1076     }
1077 }
1078 // ----- assets/Scripts/Games/OpticalPuzzle/Application/OpticalPuzzleStateMachine.ts -----
1079 /**
1080  * 与《小游戏设计文档》及 project-rules 统一的最小状态枚举。
1081  * 具体流转由 OpticalPuzzleSession 驱动。
1082  */
1083 export enum OpticalGameFlowState {
1084     BOOT = 'BOOT',
1085     READY = 'READY',
1086     RUNNING = 'RUNNING',
1087     PAUSED = 'PAUSED',
1088     SETTLEMENT = 'SETTLEMENT',
1089     FINISHED = 'FINISHED',
1090 }
1091 // ----- assets/Scripts/Games/OpticalPuzzle/Config/OpticalPuzzleLevels.ts -----
1092 import type { IOpticalLevelConfig, IOpticalLevelLayeredSource } from './OpticalPuzzleLevelSchema';
1093 import { parseLayeredGridsToLevelConfig } from './OpticalPuzzleLevelSchema';
1094 /**
1095  * 关卡源数据：每关须完整配置四层同尺寸网格（`staticLayout` / `objects` / `colors` / `directions`）。
1096  * 含义见 `OpticalPuzzleLevelSchema.ts`；层 3/4 仅在光源、目标、元件处标注，其余为 `.`。
1097  * 元件：层 2 写数字 `0`~`4`（通道类型）；层 4 `w/a/s/d` 旋转朝向。
1098  */
1099 export const OPTICAL_LEVEL_LAYERED_SOURCES: IOpticalLevelLayeredSource[] = [
1100     /*{
1101         levelId: 1,
1102         levelName: '点亮目标',

```

```

1103     perfectSteps: 2,
1104     threeStarSteps: 4,
1105     twoStarSteps: 10,
1106     oneStarSteps: 20,
1107     height: 5,
1108     width: 9,
1109     staticLayout: [
1110         '#####',
1111         '#..#..#',
1112         '#.....#',
1113         '#..#..#',
1114         '#####',
1115     ],
1116     objects: [
1117         '#####',
1118         '#..#..#',
1119         '#S..O..E#',
1120         '#..#@#..#',
1121         '#####',
1122     ],
1123     colors: [
1124         '#####',
1125         '#..#..#',
1126         '#W.W.W.W#',
1127         '#..#..#',
1128         '#####',
1129     ],
1130     directions: [
1131         '#####',
1132         '#..#..#',
1133         '#d..w...#',
1134         '#..#..#',
1135         '#####',
1136     ],
1137 },*/
1138 {
1139     levelId: 1,//第 1 关
1140     levelName: '关卡 1（难度 19）',
1141     perfectSteps: 1,
1142     threeStarSteps: 1,
1143     twoStarSteps: 3,
1144     oneStarSteps: 5,
1145     bestSolution: ['w'],
1146     height: 7,
1147     width: 9,
1148     staticLayout: [
1149         '#####',
1150         '#.###..#',
1151         '#.....#',
1152         '#.....#',
1153         '#.....#',
1154         '###.###.##',
1155         '#####',
1156     ],
1157     objects: [
1158         '#####',
1159         '#.S###E.#',
1160         '#.....#',

```

```

1161         '#.1.....#',
1162         '#.....1.#',
1163         '##.###@##',
1164         '#####',
1165     ],
1166     colors: [
1167         '#####',
1168         '#.W###W.#',
1169         '#.....#',
1170         '#.W....#',
1171         '#....W.#',
1172         '##.###.##',
1173         '#####',
1174     ],
1175     directions: [
1176         '#####',
1177         '#.s###..#',
1178         '#.....#',
1179         '#.w....#',
1180         '#.....a.#',
1181         '##.###.##',
1182         '#####',
1183     ],
1184 },
1185 {
1186     levelId: 2,//第 2 关
1187     levelName: '关卡 2（难度 18）',
1188     perfectSteps: 1,
1189     threeStarSteps: 1,
1190     twoStarSteps: 3,
1191     oneStarSteps: 5,
1192     bestSolution: ['a'],
1193     height: 6,
1194     width: 9,
1195     staticLayout: [
1196         '#####',
1197         '#.....#',
1198         '##.....##',
1199         '##.....##',
1200         '#.....#',
1201         '#####',
1202     ],
1203     objects: [
1204         '#####',
1205         '#E....1@#',
1206         '##.....##',
1207         '##.....##',
1208         '#S...1..#',
1209         '#####',
1210     ],
1211     colors: [
1212         '#####',
1213         '#W...W.#',
1214         '##.....##',
1215         '##.....##',
1216         '#W...W..#',
1217         '#####',
1218     ],

```

```

1219         directions: [
1220             '#####',
1221             '#....s.#',
1222             '##....##',
1223             '##....##',
1224             '#d...a..#',
1225             '#####',
1226         ],
1227     },
1228     {
1229         levelId: 3,//第 3 关
1230         levelName: '关卡 3（难度 23）',
1231         perfectSteps: 7,
1232         threeStarSteps: 9,
1233         twoStarSteps: 15,
1234         oneStarSteps: 30,
1235         bestSolution: ['s', 'd', 'd', 'd', 's', 's', 'a'],
1236         height: 6,
1237         width: 9,
1238         staticLayout: [
1239             '#####',
1240             '#.....#',
1241             '#.....#',
1242             '#.....#',
1243             '#.....#',
1244             '#####',
1245         ],
1246         objects: [
1247             '#####',
1248             '#..S@E..#',
1249             '#.....#',
1250             '#.....#',
1251             '#..1..1.#',
1252             '#####',
1253         ],
1254         colors: [
1255             '#####',
1256             '#..W.W..#',
1257             '#.....#',
1258             '#.....#',
1259             '#..W.W.#',
1260             '#####',
1261         ],
1262         directions: [
1263             '#####',
1264             '#..s....#',
1265             '#.....#',
1266             '#.....#',
1267             '#..w..a.#',
1268             '#####',
1269         ],
1270     },
1271     {
1272         levelId: 4,//第 4 关
1273         levelName: '关卡 4（难度 27）',
1274         perfectSteps: 6,
1275         threeStarSteps: 7,
1276         twoStarSteps: 9,

```

```

1277     oneStarSteps: 15,
1278     bestSolution: ['d', 'd', 'w', 'd', 'w', 'a'],
1279     height: 6,
1280     width: 9,
1281     staticLayout: [
1282         '#####',
1283         '#.....#',
1284         '#.....#',
1285         '#.....#',
1286         '#...#...#',
1287         '#####',
1288     ],
1289     objects: [
1290         '#####',
1291         '#..1....#',
1292         '#.....1.#',
1293         '#...@...#',
1294         '#S.1#1.E#',
1295         '#####',
1296     ],
1297     colors: [
1298         '#####',
1299         '#.W...#',
1300         '#....W.#',
1301         '#.....#',
1302         '#W.W#W.W#',
1303         '#####',
1304     ],
1305     directions: [
1306         '#####',
1307         '#.d....#',
1308         '#.....s.#',
1309         '#.....#',
1310         '#d.a#w..#',
1311         '#####',
1312     ],
1313 },
1314 {
1315     levelId: 5,//第 5 关
1316     levelName: '关卡 5（难度 25）',
1317     perfectSteps: 13,
1318     threeStarSteps: 15,
1319     twoStarSteps: 20,
1320     oneStarSteps: 30,
1321     bestSolution: [
1322         'd', 'd', 'w', 'd', 's', 'd', 's', 'a', 'a', 'a', 'a', 'a', 'w',
1323     ],
1324     height: 6,
1325     width: 10,
1326     staticLayout: [
1327         '#####',
1328         '#.....#',
1329         '#...#...#',
1330         '#.....#',
1331         '#.....#',
1332         '#####',
1333     ],
1334     objects: [

```

```

1335         '#####',
1336         '#S.....#',
1337         '#...#...#',
1338         '#..@.1...#',
1339         '#.....E#',
1340         '#####',
1341     ],
1342     colors: [
1343         '#####',
1344         '#W.....#',
1345         '#...#...#',
1346         '#...W...#',
1347         '#.....W#',
1348         '#####',
1349     ],
1350     directions: [
1351         '#####',
1352         '#s.....#',
1353         '#...#...#',
1354         '#...w...#',
1355         '#.....#',
1356         '#####',
1357     ],
1358 },
1359 {
1360     levelId: 6,//第 6 关
1361     levelName: '关卡 6（难度 31）',
1362     perfectSteps: 20,
1363     threeStarSteps: 23,
1364     twoStarSteps: 30,
1365     oneStarSteps: 40,
1366     bestSolution: [
1367         's', 's', 'd', 's', 's', 'a', 'a', 'w', 'd', 's', 'd', 'w', 'a', 'w', 'd', 'd', 'd', 'a', 'a', 's',
1368     ],
1369     height: 7,
1370     width: 10,
1371     staticLayout: [
1372         '#####',
1373         '#.....#',
1374         '#.###...#',
1375         '#.....#',
1376         '#...##...#',
1377         '#....#..#',
1378         '#####',
1379     ],
1380     objects: [
1381         '#####',
1382         '#.@...E..#',
1383         '#.1###...#',
1384         '#S.....#',
1385         '#...##...#',
1386         '#....#..#',
1387         '#####',
1388     ],
1389     colors: [
1390         '#####',
1391         '#....W..#',
1392         '#.W###...#',

```

```

1393         '#W.....#',
1394         '#...##...#',
1395         '#.....#.#',
1396         '#####',
1397     ],
1398     directions: [
1399         '#####',
1400         '#.....#',
1401         '#.a###...#',
1402         '#d.....#',
1403         '#...##...#',
1404         '#.....#.#',
1405         '#####',
1406     ],
1407 },
1408 {
1409     levelId: 7,//第 7 关
1410     levelName: '关卡 7（难度 33）',
1411     perfectSteps: 18,
1412     threeStarSteps: 20,
1413     twoStarSteps: 30,
1414     oneStarSteps: 50,
1415     bestSolution: [
1416         'd','s','a','w','a','s','s','s','s','s','d','s','a','d','d','d','d','w',
1417     ],
1418     height: 9,
1419     width: 9,
1420     staticLayout: [
1421         '#####',
1422         '#.....#',
1423         '#.....#',
1424         '#...#...#',
1425         '#.###..#',
1426         '#...#...#',
1427         '#.....#',
1428         '#.....#',
1429         '#####',
1430     ],
1431     objects: [
1432         '#####',
1433         '#S.@...E#',
1434         '#.1....#',
1435         '#...#...#',
1436         '#.###..#',
1437         '#...#...#',
1438         '#.....#',
1439         '#.....1.#',
1440         '#####',
1441     ],
1442     colors: [
1443         '#####',
1444         '#W....W#',
1445         '#.W...#',
1446         '#...#...#',
1447         '#.###..#',
1448         '#...#...#',
1449         '#.....#',
1450         '#.....W#',

```

```

1451         '#####',
1452     ],
1453     directions: [
1454         '#####',
1455         '#s.....#',
1456         '#.w....#',
1457         '#...#...#',
1458         '#.###..#',
1459         '#...#...#',
1460         '#.....#',
1461         '#....a.#',
1462         '#####',
1463     ],
1464 },
1465 {
1466     levelId: 8,//第 8 关
1467     levelName: '关卡 8（难度 36）',
1468     perfectSteps: 19,
1469     threeStarSteps: 21,
1470     twoStarSteps: 30,
1471     oneStarSteps: 50,
1472     bestSolution: [
1473         's', 's', 'd', 'd', 'd', 'd', 'w', 'w', 'a', 'a', 'a', 's', 'd', 's', 'a', 'a', 'a', 'a', 'w',
1474     ],
1475     height: 6,
1476     width: 11,
1477     staticLayout: [
1478         '#####',
1479         '#.....#',
1480         '#...#...#',
1481         '#.....#',
1482         '#...#...#',
1483         '#####',
1484     ],
1485     objects: [
1486         '#####',
1487         '#S..@....E#',
1488         '#..#1#..#',
1489         '#...1...#',
1490         '#...#...#',
1491         '#####',
1492     ],
1493     colors: [
1494         '#####',
1495         '#W.....R#',
1496         '#..#W#..#',
1497         '#...R...#',
1498         '#...#...#',
1499         '#####',
1500     ],
1501     directions: [
1502         '#####',
1503         '#s.....#',
1504         '#..#w#..#',
1505         '#...a...#',
1506         '#...#...#',
1507         '#####',
1508     ],

```

```

1509     },
1510     {
1511         levelId: 9,//第 9 关
1512         levelName: '关卡 9（难度 36）',
1513         perfectSteps: 11,
1514         threeStarSteps: 15,
1515         twoStarSteps: 24,
1516         oneStarSteps: 36,
1517         bestSolution: [
1518             'w', 'a', 'a', 'w', 'a', 's', 's', 'd', 's', 'a', 'w',
1519         ],
1520         height: 7,
1521         width: 9,
1522         staticLayout: [
1523             '#####',
1524             '#.....#',
1525             '#...#...#',
1526             '#.....#',
1527             '#...#...#',
1528             '#.....#',
1529             '#####',
1530         ],
1531         objects: [
1532             '#####',
1533             '#1.....E#',
1534             '#...#...#',
1535             '#3..1..S#',
1536             '#...#@..#',
1537             '#.....E#',
1538             '#####',
1539         ],
1540         colors: [
1541             '#####',
1542             '#R.....R#',
1543             '#...#...#',
1544             '#W.W.W.W#',
1545             '#...#...#',
1546             '#.....W#',
1547             '#####',
1548         ],
1549         directions: [
1550             '#####',
1551             '#d.....#',
1552             '#...#...#',
1553             '#d..w..a#',
1554             '#...#...#',
1555             '#.....#',
1556             '#####',
1557         ],
1558     },
1559     {
1560         levelId: 10,//第 10 关
1561         levelName: '关卡 10（难度 37）',
1562         perfectSteps: 18,
1563         threeStarSteps: 24,
1564         twoStarSteps: 30,
1565         oneStarSteps: 50,
1566         bestSolution: [

```

```

1567         'a', 'w', 'w', 's', 'a', 'a', 'a', 'w', 'a', 's', 'd', 'd', 'w', 'w', 'd', 'd', 'd', 's',
1568     ],
1569     height: 6,
1570     width: 9,
1571     staticLayout: [
1572         '#####',
1573         '#.....#',
1574         '#...#...#',
1575         '#.....#',
1576         '#.....#',
1577         '#####',
1578     ],
1579     objects: [
1580         '#####',
1581         '#1.....#',
1582         '#...#...#',
1583         '#.1.1..#',
1584         '#...E.@S#',
1585         '#####',
1586     ],
1587     colors: [
1588         '#####',
1589         '#R.....#',
1590         '#...#...#',
1591         '#.W.W..#',
1592         '#...R.W#',
1593         '#####',
1594     ],
1595     directions: [
1596         '#####',
1597         '#d.....#',
1598         '#...#...#',
1599         '#.w.s..#',
1600         '#.....w#',
1601         '#####',
1602     ],
1603 },
1604 {
1605     levelId: 11,//第 11 关
1606     levelName: '关卡 11（难度 55）',
1607     perfectSteps: 38,
1608     threeStarSteps: 44,
1609     twoStarSteps: 50,
1610     oneStarSteps: 70,
1611     bestSolution: [
1612         'd', 's', 'd', 'w', 'd', 's', 's', 's', 's', 'd', 's', 'a', 'a', 'a', 'a', 'a', 'w', 'a', 's',
1613         'd', 'w', 'w', 'w', 'w', 'd', 'd', 'd', 'w', 'd', 's', 's', 's', 's', 's', 'a', 's', 'd', 'w',
1614     ],
1615     height: 9,
1616     width: 9,
1617     staticLayout: [
1618         '#####',
1619         '#.....#',
1620         '#.....#',
1621         '#...#...#',
1622         '#.###..#',
1623         '#...#...#',
1624         '#.....#',

```

```

1625         '#.....#',
1626         '#####',
1627     ],
1628     objects: [
1629         '#####',
1630         '#S.@...E#',
1631         '#.1.1..#',
1632         '#...#...#',
1633         '#.###..#',
1634         '#...#...#',
1635         '#.....#',
1636         '#...2...#',
1637         '#####',
1638     ],
1639     colors: [
1640         '#####',
1641         '#W....R#',
1642         '#.WW..#',
1643         '#...#...#',
1644         '#.###..#',
1645         '#...#...#',
1646         '#.....#',
1647         '#...R...#',
1648         '#####',
1649     ],
1650     directions: [
1651         '#####',
1652         '#s.....#',
1653         '#.a.w..#',
1654         '#...#...#',
1655         '#.###..#',
1656         '#...#...#',
1657         '#.....#',
1658         '#...d...#',
1659         '#####',
1660     ],
1661 },
1662 {
1663     levelId: 12,//第 12 关
1664     levelName: '关卡 12（难度 52）',
1665     perfectSteps: 32,
1666     threeStarSteps: 40,
1667     twoStarSteps: 48,
1668     oneStarSteps: 60,
1669     bestSolution: [
1670         'w', 'w', 'w', 's', 'd', 'd', 'w', 'w', 'a', 'a', 'a', 'a', 'd', 's', 's', 's', 'a', 'a',
1671         'w', 'd', 'd', 'd', 'w', 's', 'd', 's', 'd', 'w', 'w', 's', 'a',
1672     ],
1673     height: 7,
1674     width: 8,
1675     staticLayout: [
1676         '#####',
1677         '#.....#',
1678         '#.#.#.#',
1679         '#.....#',
1680         '#.....#',
1681         '#.#.#.#',
1682         '#####',

```

```

1683     ],
1684     objects: [
1685         '#####',
1686         '#.....#',
1687         '#.#.3#.#',
1688         '#.11...#',
1689         '#.....#',
1690         '#E#@S#E#',
1691         '#####',
1692     ],
1693     colors: [
1694         '#####',
1695         '#.....#',
1696         '#.#.W#.#',
1697         '#.RW...#',
1698         '#.....#',
1699         '#W#.W#R#',
1700         '#####',
1701     ],
1702     directions: [
1703         '#####',
1704         '#.....#',
1705         '#.#.s#.#',
1706         '#.sd...#',
1707         '#.....#',
1708         '#.#.w#.#',
1709         '#####',
1710     ],
1711 },
1712 {
1713     levelId: 13,//第 13 关
1714     levelName: '关卡 13（难度 54）',
1715     perfectSteps: 11,
1716     threeStarSteps: 12,
1717     twoStarSteps: 16,
1718     oneStarSteps: 30,
1719     bestSolution: [
1720         'w', 'w', 'a', 's', 'a', 'w', 'd', 'w', 'd', 'd', 'w',
1721     ],
1722     height: 7,
1723     width: 9,
1724     staticLayout: [
1725         '#####',
1726         '#.....#',
1727         '#.....#',
1728         '#.....#',
1729         '#.....#',
1730         '#.....#',
1731         '#####',
1732     ],
1733     objects: [
1734         '#####',
1735         '#.E...E#',
1736         '#.....#',
1737         '#..33...#',
1738         '#.....#',
1739         '#.S.@.E#',
1740         '#####',

```

```

20997     private _ensureTopBarIconButtons(): void {
20998         const topBar = this.node.getChildByName('layHub')?.getChildByName('TopBar') ?? null;
20999         if (!this.btnBackMenu?.isValid) {
21000             this.btnBackMenu = topBar?.getChildByName('BtnBackMenu') ?? null;
21001         }
21002         if (!this.btnLevelSelect?.isValid) {
21003             this.btnLevelSelect = topBar?.getChildByName('BtnLevelSelect') ?? null;
21004         }
21005         ensureBackButtonView(this.btnBackMenu);
21006         ensureLevelSelectButtonView(this.btnLevelSelect);
21007         ensureStepViews(topBar);
21008     }
21009     /** 未在检查器绑定时，按 layerOverlay/answerPanel 解析 */
21010     private _resolveAnswerPanel(): void {
21011         if (this.answerPanel?.isValid) {
21012             return;
21013         }
21014         this.answerPanel = resolveAnswerPanelNode(this.node);
21015     }
21016     /** 未在检查器绑定时，按 GameRoot/layHub/TopBar/LabelLevel 解析 */
21017     private _resolveLabelLevel(): void {
21018         if (this.labelLevel?.isValid) {
21019             return;
21020         }
21021         const labelNode =
21022             this.node.getChildByName('layHub')?.getChildByName('TopBar')?.getChildByName('LabelLevel') ??
21023             null;
21024         if (!labelNode?.isValid) {
21025             return;
21026         }
21027         this.labelLevel = labelNode.getComponent(Label);
21028     }
21029     private _onOpticalSnapshotChanged(payload: OpticalSnapshotNotify): void {
21030         const snap = payload?.snapshot;
21031         if (!snap) {
21032             return;
21033         }
21034         if (snap.levelId !== this._displayedLevelId) {
21035             this.refreshLevelLabel(snap.levelId);
21036         }
21037         syncLevelSelectPanelVisuals(this.levelSelectPanel);
21038     }
21039 }
21040 // ----- assets/Scripts/Manager/LevelManager.ts -----
21041 import { _decorator, Component } from 'cc';
21042 const { ccclass } = _decorator;
21043 /** 关卡列表与切换占位；光学解谜关卡整数 id 与 DataManager.opticalCurrentLevelId 对齐 */
21044 @ccclass('LevelManager')
21045 export class LevelManager extends Component {}
21046 // ----- assets/Scripts/Manager/MenuManager.ts -----
21047 import { _decorator, Button, Component, director, Node } from 'cc';
21048 import {
21049     ensureAboutMePanel,
21050     resolveAboutMePanelNode,
21051 } from '../Games/OpticalPuzzle/Presentation/OpticalPuzzleAboutMePanel';
21052 import {
21053     ensureLevelSelectPanel,
21054     openLevelSelectPanel,

```

```

21055     prewarmLevelSelectPanel,
21056     resolveLevelSelectPanelNode,
21057 } from '../Games/OpticalPuzzle/Presentation/OpticalPuzzleLevelSelectPanel';
21058 import { ensureMenuGameTitleView } from '../MenuGameTitleView';
21059 import { ensureMenuGameTitle2View } from '../MenuGameTitle2View';
21060 import { ensureMenuGameTitleIconView } from '../MenuGameTitleIconView';
21061 import { ensureMenuGameTitleIcon2View } from '../MenuGameTitleIcon2View';
21062 import { ensureMenuAboutButtonView } from '../MenuAboutButtonView';
21063 import { ensureMenuLevelSelectButtonView } from '../MenuLevelSelectButtonView';
21064 import { ensureMenuShareButtonView } from '../MenuShareButtonView';
21065 import { ensureMenuStartButtonView } from '../MenuStartButtonView';
21066 import { DataManager } from './DataManager';
21067 import { AUDIO_EFFECT_ENUM, EVENT_ENUM, SCENE_ENUM } from './Utils/Enum';
21068 import { PLAY_AUDIO } from './Utils/Event';
21069 import { invokeWeChatFriendShare } from './Utils/WeChatShare';
21070 const { ccclass, property } = _decorator;
21071 /**
21072  * 菜单场景编排：读档、开始游戏、选关弹层等。
21073  * 挂在 MenuRoot；按钮拖入对应属性，勿在 Button Click Events 里重复绑定逻辑。
21074  */
21075 @ccclass('MenuManager')
21076 export class MenuManager extends Component {
21077     /** 开始游戏按钮（MainPanel/BtnStart） */
21078     @property(Node)
21079     btnStart: Node = null;
21080     /** 打开选关面板（MainPanel/BtnLevelSelect） */
21081     @property(Node)
21082     btnLevelSelect: Node = null;
21083     /** 选关弹层根节点（layerOverlay/LevelSelectPanel） */
21084     @property(Node)
21085     levelSelectPanel: Node = null;
21086     /** 微信分享（MainPanel/BtnShare） */
21087     @property(Node)
21088     btnShare: Node = null;
21089     /** 关于（MainPanel/BtnAbout） */
21090     @property(Node)
21091     btnAbout: Node = null;
21092     /** 关于弹层（layerOverlay/aboutMePanel） */
21093     @property(Node)
21094     aboutMePanel: Node = null;
21095     /** 游戏标题（MainPanel/GameTitle，矢量描边） */
21096     @property(Node)
21097     gameTitle: Node = null;
21098     /** 副标题（MainPanel/GameTitle2，矢量描边） */
21099     @property(Node)
21100     gameTitle2: Node = null;
21101     /** 标题主角图标（MainPanel/GameTitleIcon，睁眼朝左看） */
21102     @property(Node)
21103     gameTitleIcon: Node = null;
21104     /** 标题白色光源（MainPanel/GameTitleIcon2，朝右 1000px 光束） */
21105     @property(Node)
21106     gameTitleIcon2: Node = null;
21107     protected onLoad(): void {
21108         DataManager.instance.init();
21109         director.preloadScene(SCENE_ENUM.GAME);
21110         this._resolveMenuNodes();
21111         ensureMenuStartButtonView(this.btnStart);
21112         ensureMenuLevelSelectButtonView(this.btnLevelSelect);

```

```

21113         ensureMenuShareButtonView(this.btnShare);
21114         ensureMenuAboutButtonView(this.btnAbout);
21115         ensureLevelSelectPanel(this.levelSelectPanel);
21116         ensureAboutMePanel(this.aboutMePanel);
21117         ensureMenuGameTitleView(this.gameTitle);
21118         ensureMenuGameTitle2View(this.gameTitle2);
21119         ensureMenuGameTitleIconView(this.gameTitleIcon);
21120         ensureMenuGameTitleIcon2View(this.gameTitleIcon2);
21121         this.closeLevelSelect();
21122         this.closeAbout();
21123         prewarmLevelSelectPanel(this.levelSelectPanel);
21124         this.bindStartButton();
21125         this.bindLevelSelectButton();
21126         this.bindShareButton();
21127         this.bindAboutButton();
21128     }
21129     protected onDestroy(): void {
21130         this.unbindStartButton();
21131         this.unbindLevelSelectButton();
21132         this.unbindShareButton();
21133         this.unbindAboutButton();
21134     }
21135     /** 进入局内场景（当前进度关卡由 DataManager.opticalCurrentLevelId 决定） */
21136     startGame(): void {
21137         director.loadScene(SCENE_ENUM.GAME);
21138     }
21139     /** 显示选关面板 */
21140     openLevelSelect(): void {
21141         openLevelSelectPanel(this.levelSelectPanel);
21142     }
21143     /** 隐藏选关面板（MenuOverlayWindow 关闭时也可调） */
21144     closeLevelSelect(): void {
21145         if (!this.levelSelectPanel?.isValid) {
21146             return;
21147         }
21148         this.levelSelectPanel.active = false;
21149     }
21150     /** 显示关于弹层 */
21151     openAbout(): void {
21152         if (!this.aboutMePanel?.isValid) {
21153             return;
21154         }
21155         this.closeLevelSelect();
21156         this.aboutMePanel.active = true;
21157     }
21158     /** 隐藏关于弹层 */
21159     closeAbout(): void {
21160         if (!this.aboutMePanel?.isValid) {
21161             return;
21162         }
21163         this.aboutMePanel.active = false;
21164     }
21165     /** 未在检查器绑定时，按 MenuRoot 子节点名解析 */
21166     private _resolveMenuNodes(): void {
21167         if (!this.btnStart?.isValid) {
21168             this.btnStart = this._findMainPanel()?.getChildByName('BtnStart') ?? null;
21169         }
21170         if (!this.btnLevelSelect?.isValid) {

```

```

21171         this.btnLevelSelect = this._findMainPanel()?.getChildByName('BtnLevelSelect') ?? null;
21172     }
21173     if (!this.btnShare?.isValid) {
21174         this.btnShare = this._findMainPanel()?.getChildByName('BtnShare') ?? null;
21175     }
21176     if (!this.btnAbout?.isValid) {
21177         this.btnAbout = this._findMainPanel()?.getChildByName('BtnAbout') ?? null;
21178     }
21179     if (!this.levelSelectPanel?.isValid) {
21180         this.levelSelectPanel = resolveLevelSelectPanelNode(this.node);
21181     }
21182     if (!this.aboutMePanel?.isValid) {
21183         this.aboutMePanel = resolveAboutMePanelNode(this.node);
21184     }
21185     if (!this.gameTitle?.isValid) {
21186         this.gameTitle = this._findMainPanel()?.getChildByName('GameTitle') ?? null;
21187     }
21188     if (!this.gameTitle2?.isValid) {
21189         this.gameTitle2 = this._findMainPanel()?.getChildByName('GameTitle2') ?? null;
21190     }
21191     if (!this.gameTitleIcon?.isValid) {
21192         this.gameTitleIcon = this._findMainPanel()?.getChildByName('GameTitleIcon') ?? null;
21193     }
21194     if (!this.gameTitleIcon2?.isValid) {
21195         this.gameTitleIcon2 = this._findMainPanel()?.getChildByName('GameTitleIcon2') ?? null;
21196     }
21197 }
21198 private _findMainPanel(): Node | null {
21199     return this.node.getChildByName('layerMain')?.getChildByName('MainPanel') ?? null;
21200 }
21201 private bindStartButton(): void {
21202     this.bindNodeClick(this.btnStart, this.onStartClick);
21203 }
21204 private unbindStartButton(): void {
21205     this.unbindNodeClick(this.btnStart, this.onStartClick);
21206 }
21207 private bindLevelSelectButton(): void {
21208     this.bindNodeClick(this.btnLevelSelect, this.onLevelSelectClick);
21209 }
21210 private unbindLevelSelectButton(): void {
21211     this.unbindNodeClick(this.btnLevelSelect, this.onLevelSelectClick);
21212 }
21213 private bindShareButton(): void {
21214     this.bindNodeClick(this.btnShare, this.onShareClick);
21215 }
21216 private unbindShareButton(): void {
21217     this.unbindNodeClick(this.btnShare, this.onShareClick);
21218 }
21219 private bindAboutButton(): void {
21220     this.bindNodeClick(this.btnAbout, this.onAboutClick);
21221 }
21222 private unbindAboutButton(): void {
21223     this.unbindNodeClick(this.btnAbout, this.onAboutClick);
21224 }
21225 private bindNodeClick(node: Node | null, handler: () => void): void {
21226     if (!node?.isValid) {
21227         return;
21228     }

```

```

21229         const button = node.getComponent(Button);
21230         if (button) {
21231             button.node.on(Button.EventType.CLICK, handler, this);
21232             return;
21233         }
21234         node.on(Node.EventType.TOUCH_END, handler, this);
21235     }
21236     private unbindNodeClick(node: Node | null, handler: () => void): void {
21237         if (!node?.isValid) {
21238             return;
21239         }
21240         const button = node.getComponent(Button);
21241         if (button) {
21242             button.node.off(Button.EventType.CLICK, handler, this);
21243             return;
21244         }
21245         node.off(Node.EventType.TOUCH_END, handler, this);
21246     }
21247     private onStartClick(): void {
21248         PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
21249         this.startGame();
21250     }
21251     private onLevelSelectClick(): void {
21252         PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
21253         this.openLevelSelect();
21254     }
21255     /** 微信好友分享（非微信环境 onFail） */
21256     private onShareClick(): void {
21257         invokeWeChatFriendShare({
21258             onSuccess: () => {
21259                 /* 可按需加分享奖励 */
21260             },
21261             onFail: () => {
21262                 /* 浏览器预览等环境无 wx.shareAppMessage */
21263             },
21264         });
21265         PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
21266     }
21267     private onAboutClick(): void {
21268         PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
21269         this.openAbout();
21270     }
21271 }
21272 // ----- assets/Scripts/Manager/MusicManager.ts -----
21273 import {
21274     _decorator,
21275     AudioClip,
21276     AudioSource,
21277     Component,
21278     director,
21279     Director,
21280     Node,
21281 } from 'cc';
21282 import { DataManager } from './DataManager';
21283 import { AUDIO_EFFECT_ENUM, EVENT_ENUM, SCENE_ENUM } from '../Utils/Enum';
21284 import { PLAY_AUDIO, PLAY_BGM } from '../Utils/Event';
21285 import {
21286     cancelWeChatRewardedVideoPreloadSchedule,

```

```

21287     scheduleWeChatRewardedVideoPreloadForGame,
21288 } from './Utils/WeChatRewardedVideoAd';
21289 import { initWeChatMiniGameShare, trySettlePendingShareAfterWxOnShow } from './Utils/WeChatShare';
21290 const { ccclass, property } = _decorator;
21291 /**
21292  * 音频管理: PersistRoot + 事件驱动 (project-rules)。
21293  * 全局 BGM (如 Bg.mp3) 在 Menu / Game 场景循环播放; 音效经 PLAY_AUDIO 播放。
21294  * 未绑定 AudioClip 或 AudioSource 时跳过, 避免空引用。
21295  */
21296 @ccclass('MusicManager')
21297 export class MusicManager extends Component {
21298     /**#region 编辑器绑定
21299     /** 全局背景音乐 (如 assets/Audio/Bg.mp3, Menu / Game 共用) */
21300     @property({ type: AudioClip, tooltip: '全局 BGM (Bg.mp3)' })
21301     bgm: AudioClip | null = null;
21302     /** UI 通用按钮点击 (建议 assets/Audio/Click.mp3) */
21303     @property({ type: AudioClip, tooltip: '通用按钮点击 (Click.mp3)' })
21304     clickButton: AudioClip | null = null;
21305     /** 四向键移动成功: 主角走一格或推动元件 (建议 Right.mp3) */
21306     @property({ type: AudioClip, tooltip: '四向键移动成功 (Right.mp3)' })
21307     opticalMoveSuccess: AudioClip | null = null;
21308     /** 四向键移动失败: 与主角阻拦脸同级 (建议 Failed.mp3) */
21309     @property({ type: AudioClip, tooltip: '四向键移动失败 (Failed.mp3)' })
21310     opticalMoveFail: AudioClip | null = null;
21311     /** 单个目标灯被点亮 (待加资源, 如 Light.mp3) */
21312     @property({ type: AudioClip, tooltip: '单个目标灯点亮 (待加 Light.mp3 等)' })
21313     opticalTargetLit: AudioClip | null = null;
21314     /** 关卡通关成功 (建议 Pass.mp3; winPanel 展示时播放, 前有 PRE_WIN_PANEL_DELAY_SEC) */
21315     @property({ type: AudioClip, tooltip: '关卡通关 (Pass.mp3; 结算 winPanel 弹出时播放)' })
21316     opticalLevelComplete: AudioClip | null = null;
21317     /** 用于循环 BGM; 音效使用 playOneShot 播在同一 AudioSource 上 */
21318     @property(AudioSource)
21319     audioSource: AudioSource | null = null;
21320     /**#endregion
21321     private static _persistRegistered = false;
21322     /** 当前正在播放的 BGM Clip, 避免同曲重复 stop/play */
21323     private _currentBgmClip: AudioClip | null = null;
21324     private readonly _wxOnShowHandler = (): void => {
21325         trySettlePendingShareAfterWxOnShow();
21326     };
21327     protected onLoad(): void {
21328         this.registerPersistRoot();
21329         director.on(Director.EVENT_AFTER_SCENE_LAUNCH, this.onAfterSceneLaunch, this);
21330         PLAY_AUDIO.on(EVENT_ENUM.PLAY_AUDIO, this.onAudioPlay, this);
21331         PLAY_BGM.on(EVENT_ENUM.PLAY_BGM, this.onPlayBgmEvent, this);
21332         initWeChatMiniGameShare();
21333         this._bindWeChatOnShow();
21334     }
21335     protected onDestroy(): void {
21336         this._unbindWeChatOnShow();
21337         cancelWeChatRewardedVideoPreloadSchedule();
21338         director.off(Director.EVENT_AFTER_SCENE_LAUNCH, this.onAfterSceneLaunch, this);
21339         PLAY_AUDIO.off(EVENT_ENUM.PLAY_AUDIO, this.onAudioPlay, this);
21340         PLAY_BGM.off(EVENT_ENUM.PLAY_BGM, this.onPlayBgmEvent, this);
21341     }
21342     /**#region BGM
21343     onAfterSceneLaunch(): void {
21344         const sceneName = director.getScene()?.name ?? '';

```

```

21345         this.switchBgmBySceneName(sceneName);
21346         cancelWeChatRewardedVideoPreloadSchedule();
21347         if (sceneName === SCENE_ENUM.GAME) {
21348             scheduleWeChatRewardedVideoPreloadForGame();
21349         }
21350     }
21351     /** 设置页切换 BGM 开关后可 emit PLAY_BGM 刷新当前 BGM */
21352     onPlayBgmEvent(): void {
21353         const sceneName = director.getScene()?.name ?? '';
21354         this.switchBgmBySceneName(sceneName);
21355     }
21356     switchBgmBySceneName(sceneName: string): void {
21357         if (sceneName === SCENE_ENUM.MENU || sceneName === SCENE_ENUM.GAME) {
21358             this.playGlobalBgm();
21359         } else {
21360             this.stopBgm();
21361         }
21362     }
21363     playGlobalBgm(): void {
21364         if (!DataManager.instance.bgmOn) {
21365             this.stopBgm();
21366             return;
21367         }
21368         this.playBgmClip(this.bgm);
21369     }
21370     private playBgmClip(clip: AudioClip | null): void {
21371         const source = this.audioSource;
21372         if (!source) {
21373             return;
21374         }
21375         if (!clip) {
21376             this.stopBgm();
21377             return;
21378         }
21379         if (this._currentBgmClip === clip && source.playing) {
21380             return;
21381         }
21382         source.stop();
21383         source.clip = clip;
21384         source.loop = true;
21385         source.play();
21386         this._currentBgmClip = clip;
21387     }
21388     private stopBgm(): void {
21389         if (this.audioSource) {
21390             this.audioSource.stop();
21391         }
21392         this._currentBgmClip = null;
21393     }
21394     //endregion
21395     //region 音效
21396     onAudioPlay(type: AUDIO_EFFECT_ENUM | AUDIO_EFFECT_ENUM[]): void {
21397         if (!DataManager.instance.sfxOn || !this.audioSource) {
21398             return;
21399         }
21400         const types = Array.isArray(type) ? type : [type];
21401         for (const sfxType of types) {
21402             const clip = this.resolveSfxClip(sfxType);

```

```

21403         if (clip) {
21404             this.audioSource.playOneShot(clip);
21405         }
21406     }
21407 }
21408 private resolveSfxClip(type: AUDIO_EFFECT_ENUM): AudioClip | null {
21409     switch (type) {
21410         case AUDIO_EFFECT_ENUM.CLICK_BUTTON:
21411             return this.clickButton;
21412         case AUDIO_EFFECT_ENUM.OPTICAL_MOVE_SUCCESS:
21413             return this.opticalMoveSuccess;
21414         case AUDIO_EFFECT_ENUM.OPTICAL_MOVE_FAIL:
21415             return this.opticalMoveFail;
21416         case AUDIO_EFFECT_ENUM.OPTICAL_TARGET_LIT:
21417             return this.opticalTargetLit;
21418         case AUDIO_EFFECT_ENUM.OPTICAL_LEVEL_COMPLETE:
21419             return this.opticalLevelComplete;
21420         default:
21421             return null;
21422     }
21423 }
21424 //endregion
21425 /** 持久化 PersistRoot (含 AudioService / ToastService 兄弟节点) */
21426 private registerPersistRoot(): void {
21427     if (MusicManager._persistRegistered) {
21428         return;
21429     }
21430     const root = this.findPersistRootNode();
21431     director.addPersistRootNode(root);
21432     MusicManager._persistRegistered = true;
21433 }
21434 private findPersistRootNode(): Node {
21435     let node: Node | null = this.node;
21436     while (node) {
21437         if (node.name === 'PersistRoot') {
21438             return node;
21439         }
21440         node = node.parent;
21441     }
21442     return this.node;
21443 }
21444 private _bindWeChatOnShow(): void {
21445     const wxApi = (globalThis as { wx?: { onShow?(fn: () => void): void; offShow?(fn: () => void): void } }).wx;
21446     wxApi?.onShow?.(this._wxOnShowHandler);
21447 }
21448 private _unbindWeChatOnShow(): void {
21449     const wxApi = (globalThis as { wx?: { offShow?(fn: () => void): void } }).wx;
21450     wxApi?.offShow?.(this._wxOnShowHandler);
21451 }
21452 }
21453 // ----- assets/Scripts/Manager/ToastManager.ts -----
21454 import { _decorator, Component, Prefab, instantiate, Node, Label, UITransform, UIOpacity, tween, Tween } from
21455 'cc';
21456 import { RoundBox } from '../RoundBox';
21457 const { cclass, property } = _decorator;
21458 /**
21459  * Toast: 展示文案 + 背景宽度; 显示 DISPLAY_SEC 秒后, FADE_SEC 秒内透明度变为 0。
21460  * 新一次 show 会立刻打断上一次动画并展示新内容。

```

```

21461 * 预制体根节点下需有子节点: text (挂 Label)、bg (挂 UITransform 可调宽度)。
21462 */
21463 @ccclass('ToastManager')
21464 export class ToastManager extends Component {
21465     // Toast 预制体 (根节点下子节点: text、bg)
21466     @property(Prefab)
21467     toastPrefab: Prefab = null;
21468     private static readonly DISPLAY_SEC = 1;
21469     private static readonly FADE_SEC = 1;
21470     /** 未传或非法 bgWidth 时使用 (避免 undefined → NaN 导致 UITransform.width 无效) */
21471     private static readonly DEFAULT_BG_WIDTH = 320;
21472     private _toastRoot: Node | null = null;
21473     private static resolveBgWidth(bgWidth: number | undefined): number {
21474         if (typeof bgWidth !== 'number' || !Number.isFinite(bgWidth)) {
21475             return ToastManager.DEFAULT_BG_WIDTH;
21476         }
21477         return Math.max(0, bgWidth);
21478     }
21479     private static resolveLocalY(localY: number | undefined): number {
21480         if (typeof localY !== 'number' || !Number.isFinite(localY)) {
21481             return 0;
21482         }
21483         return localY;
21484     }
21485     /**
21486      * @param message 提示文本
21487      * @param bgWidth 背景节点 `bg` 的 **宽度** (像素)。注意第二参是宽度不是 Y; 若误传
21488      * `undefined` (如可选链未绑定到值) 会走默认宽 320。
21489      * @param localY 弹窗根节点本地坐标 **Y**；省略或非法时为 0。要只调纵向位置请写 `show(msg,
21490      * undefined, 450)` 或先传宽度再传 Y。
21491      */
21492     show(message: string, bgWidth?: number, localY: number = 0): void {
21493         if (!this.toastPrefab) {
21494             console.warn('[ToastManager] 未绑定 toastPrefab');
21495             return;
21496         }
21497         if (!this._toastRoot || !this._toastRoot.isValid) {
21498             this._toastRoot = instantiate(this.toastPrefab);
21499             this._toastRoot.setParent(this.node);
21500         }
21501         const root = this._toastRoot;
21502         const py = ToastManager.resolveLocalY(localY);
21503         root.setPosition(0, py, 0);
21504         const uiOp = root.getComponent(UIOpacity) ?? root.addComponent(UIOpacity);
21505         Tween.stopAllByTarget(uiOp);
21506         /** 先定 bg 宽并立刻刷新 RoundBox: 仅改 UITransform 时 onSizeChanged 不会重算 UV, 若只
21507         schedule 下一帧强刷, 首帧仍会按旧网格绘制 → 背景宽度闪一下 */
21508         const bgNode = root.getChildByName('bg');
21509         const bgUt = bgNode?.getComponent(UITransform);
21510         const wResolved = ToastManager.resolveBgWidth(bgWidth);
21511         if (bgUt) {
21512             bgUt.width = wResolved;
21513             const rb = bgNode?.getComponent(RoundBox);
21514             if (rb?.isValid) {
21515                 rb.forceRefreshRender();
21516                 /** 首帧 chunk/texture 未就绪时补一次 (与 MenuOverlayWindow 等场景一致) */
21517                 this.scheduleOnce(() => {
21518                     if (rb.isValid && bgUt.isValid) {

```

```

21519         rb.forceRefreshRender();
21520     }
21521     }, 0);
21522 }
21523 } else {
21524     console.warn('[ToastManager] 预制体根下未找到子节点 bg 或未挂 UITransform');
21525 }
21526 const textNode = root.getChildByName('text');
21527 const label = textNode?.getComponent(Label);
21528 if (label) {
21529     label.string = message;
21530 } else {
21531     console.warn('[ToastManager] 预制体根下未找到子节点 text 或未挂 Label');
21532 }
21533 root.active = true;
21534 uiOp.opacity = 255;
21535 tween(uiOp)
21536     .delay(ToastManager.DISPLAY_SEC)
21537     .to(ToastManager.FADE_SEC, { opacity: 0 })
21538     .call(() => {
21539         // 下一帧再关节点, 避免与最后一帧 UI 提交叠在同一时刻, 圆角 VERTEX 底图容易「到
21540 0 突然没」
21541         this.scheduleOnce(() => {
21542             if (root?.isValid) {
21543                 root.active = false;
21544             }
21545             }, 0);
21546         })
21547         .start();
21548     }
21549 }
21550 // ----- assets/Scripts/MenuAboutButtonView.ts -----
21551 import {
21552     _decorator,
21553     Button,
21554     Component,
21555     Graphics,
21556     Node,
21557     UITransform,
21558 } from 'cc';
21559 import { HudButtonPressController } from
21560 './Games/OpticalPuzzle/Presentation/OpticalPuzzleHudButtonPressController';
21561 import { drawMenuAboutButtonGlyph } from './Games/OpticalPuzzle/Presentation/MenuAboutButtonGlyph';
21562 const { ccclass } = _decorator;
21563 /** 与开始键 Button.zoomScale 一致 */
21564 const ABOUT_MENU_BUTTON_ZOOM_SCALE = 0.95;
21565 /** Menu/MainPanel/BtnAbout: 圆形键帽 + 用户 icon (点击由 MenuManager 绑定) */
21566 @ccclass('MenuAboutButtonView')
21567 export class MenuAboutButtonView extends Component {
21568     private _graphics: Graphics | null = null;
21569     private _pressed = false;
21570     private _pressCtrl: HudButtonPressController | null = null;
21571     protected onLoad(): void {
21572         this._hidePlaceholderSplash();
21573         this._ensureButton();
21574         this._pressCtrl = new HudButtonPressController(this.node, (pressed) => {
21575             this._pressed = pressed;
21576             this._redraw();

```

```

21577         });
21578         this._redraw();
21579     }
21580     protected onEnable(): void {
21581         this._pressCtrl?.bind();
21582         this._redraw();
21583     }
21584     protected onDisable(): void {
21585         this._pressCtrl?.unbind();
21586         this._pressed = false;
21587     }
21588     protected onDestroy(): void {
21589         this._pressCtrl?.unbind();
21590     }
21591     private _hidePlaceholderSplash(): void {
21592         const splash = this.node.getChildByName('SpriteSplash') ?? this.node.getChildByName('bg');
21593         if (splash?.isValid) {
21594             splash.active = false;
21595         }
21596     }
21597     private _ensureButton(): void {
21598         let btn = this.getComponent(Button);
21599         if (!btn) {
21600             btn = this.addComponent(Button);
21601             btn.transition = Button.Transition.SCALE;
21602         }
21603         btn.zoomScale = ABOUT_MENU_BUTTON_ZOOM_SCALE;
21604         btn.transition = Button.Transition.SCALE;
21605     }
21606     private _ensureGraphics(): Graphics | null {
21607         if (!this._graphics?.isValid) {
21608             this._graphics = this.getComponent(Graphics) ?? this.addComponent(Graphics);
21609         }
21610         return this._graphics;
21611     }
21612     private _redraw(): void {
21613         const g = this._ensureGraphics();
21614         const ut = this.getComponent(UITransform);
21615         if (!g || !ut) {
21616             return;
21617         }
21618         g.clear();
21619         const w = ut.width;
21620         const h = ut.height;
21621         const left = -ut.anchorX * w;
21622         const bottom = -ut.anchorY * h;
21623         drawMenuAboutButtonGlyph(g, left, bottom, w, h, this._pressed);
21624     }
21625 }
21626 /** 为 MainPanel/BtnAbout 挂上键帽绘制与按压缩放 */
21627 export function ensureMenuAboutButtonView(btnAbout: Node | null): void {
21628     if (!btnAbout?.isValid) {
21629         return;
21630     }
21631     if (!btnAbout.getComponent(MenuAboutButtonView)) {
21632         btnAbout.addComponent(MenuAboutButtonView);
21633     }
21634 }

```

```
21635 // ----- assets/Scripts/MenuGameTitle2View.ts -----
21636 import { _decorator, Component, Graphics, Node, Sprite, UITransform } from 'cc';
21637 import { drawGameSubtitleOutline } from './Games/OpticalPuzzle/Presentation/MenuGameSubtitleGlyph';
21638 const { ccclass } = _decorator;
21639 /** Menu/MainPanel/GameTitle2: 副标题矢量描边（替代 PNG Sprite） */
21640 @ccclass('MenuGameTitle2View')
21641 export class MenuGameTitle2View extends Component {
21642     private _graphics: Graphics | null = null;
21643     protected onLoad(): void {
21644         this._disableRasterTitle();
21645         this._ensureGraphics();
21646         this._redraw();
21647     }
21648     protected onEnable(): void {
21649         this._redraw();
21650     }
21651     private _disableRasterTitle(): void {
21652         const splash = this.node.getChildByName('SpriteSplash');
21653         if (splash?.isValid) {
21654             splash.active = false;
21655         }
21656         const sprite = this.getComponent(Sprite);
21657         if (sprite) {
21658             sprite.enabled = false;
21659         }
21660     }
21661     private _ensureGraphics(): Graphics | null {
21662         if (!this._graphics?.isValid) {
21663             this._graphics = this.getComponent(Graphics) ?? this.addComponent(Graphics);
21664         }
21665         return this._graphics;
21666     }
21667     private _redraw(): void {
21668         const g = this._ensureGraphics();
21669         const ut = this.getComponent(UITransform);
21670         if (!g || !ut) {
21671             return;
21672         }
21673         g.clear();
21674         const w = ut.width;
21675         const h = ut.height;
21676         const left = -ut.anchorX * w;
21677         const bottom = -ut.anchorY * h;
21678         drawGameSubtitleOutline(g, left, bottom, w, h);
21679     }
21680 }
21681 /** 为 GameTitle2 节点挂上矢量副标题绘制 */
21682 export function ensureMenuGameTitle2View(titleNode: Node | null): void {
21683     if (!titleNode?.isValid) {
21684         return;
21685     }
21686     if (!titleNode.getComponent(MenuGameTitle2View)) {
21687         titleNode.addComponent(MenuGameTitle2View);
21688     }
21689 }
21690 // ----- assets/Scripts/MenuGameTitleIcon2View.ts -----
21691 import {
21692     _decorator,
```

```

21693     Component,
21694     Graphics,
21695     Node,
21696     ParticleSystem2D,
21697     Sprite,
21698     SpriteFrame,
21699     UITransform,
21700 } from 'cc';
21701 import {
21702     ensureBeamSparkSpritesLoaded,
21703     registerBeamSparkSpriteFrames,
21704 } from './Games/OpticalPuzzle/Presentation/OpticalPuzzleBeamImpactView';
21705 import {
21706     computeMenuTitleRightBeamLayout,
21707     configureMenuTitleBeamSparkParticle,
21708     drawMenuTitleWhiteSourceRightBeam,
21709 } from './Games/OpticalPuzzle/Presentation/MenuGameTitleIcon2Glyph';
21710 const { ccclass, property } = _decorator;
21711 /** Menu/MainPanel/GameTitleIcon2: 白色光源朝右，碰屏右缘 beam_spark */
21712 @ccclass('MenuGameTitleIcon2View')
21713 export class MenuGameTitleIcon2View extends Component {
21714     /** 光束右边界参考（默认向上找 Canvas） */
21715     @property({ type: Node, tooltip: '光束右边界参考节点，默认 Canvas' })
21716     beamBoundsNode: Node | null = null;
21717     /** 可选：拖入 beam_spark 贴图保活（微信包） */
21718     @property({ type: [SpriteFrame], tooltip: '可选，Sprites/OpticalFX/beam_spark_' })
21719     sparkSpriteFrames: SpriteFrame[] = [];
21720     private _graphics: Graphics | null = null;
21721     private _sparkNode: Node | null = null;
21722     private _sparkPs: ParticleSystem2D | null = null;
21723     private _sparkReady = false;
21724     protected onLoad(): void {
21725         this._disableRasterPlaceholder();
21726         this._ensureGraphics();
21727         if (registerBeamSparkSpriteFrames(this.sparkSpriteFrames)) {
21728             this._sparkReady = true;
21729         }
21730         ensureBeamSparkSpritesLoaded(() => {
21731             if (!this.node?.isValid) {
21732                 return;
21733             }
21734             this._sparkReady = true;
21735             this._ensureSparkEmitter();
21736             this._redraw();
21737         });
21738         this._redraw();
21739     }
21740     protected onEnable(): void {
21741         this._redraw();
21742     }
21743     private _disableRasterPlaceholder(): void {
21744         const splash = this.node.getChildByName('SpriteSplash');
21745         if (splash?.isValid) {
21746             splash.active = false;
21747         }
21748         const sprite = this.getComponent(Sprite);
21749         if (sprite) {
21750             sprite.enabled = false;

```

```

21751     }
21752   }
21753   private _ensureGraphics(): Graphics | null {
21754     if (!this._graphics?.isValid) {
21755       this._graphics = this.getComponent(Graphics) ?? this.addComponent(Graphics);
21756     }
21757     return this._graphics;
21758   }
21759   private _resolveBeamBoundsNode(): Node | null {
21760     if (this.beamBoundsNode?.isValid) {
21761       return this.beamBoundsNode;
21762     }
21763     let node: Node | null = this.node;
21764     while (node) {
21765       if (node.name === 'Canvas') {
21766         return node;
21767       }
21768       node = node.parent;
21769     }
21770     return null;
21771   }
21772   private _ensureSparkEmitter(): void {
21773     if (!this._sparkReady || !this._sparkPs?.isValid) {
21774       return;
21775     }
21776     const node = new Node('BeamSpark');
21777     node.setParent(this.node);
21778     const ps = node.addComponent(ParticleSystem2D);
21779     configureMenuTitleBeamSparkParticle(ps);
21780     this._sparkNode = node;
21781     this._sparkPs = ps;
21782     this.scheduleOnce(() => {
21783       if (ps?.isValid) {
21784         ps.resetSystem();
21785       }
21786     }, 0);
21787   }
21788   private _syncSpark(layout: { endX: number; endY: number; hitsScreenEdge: boolean }): void {
21789     if (!this._sparkReady) {
21790       return;
21791     }
21792     this._ensureSparkEmitter();
21793     const sparkNode = this._sparkNode;
21794     const ps = this._sparkPs;
21795     if (!sparkNode?.isValid || !ps?.isValid) {
21796       return;
21797     }
21798     if (!layout.hitsScreenEdge) {
21799       sparkNode.active = false;
21800       ps.stopSystem();
21801       return;
21802     }
21803     sparkNode.active = true;
21804     sparkNode.setPosition(layout.endX, layout.endY, 0);
21805   }
21806   private _redraw(): void {
21807     const g = this._ensureGraphics();
21808     const ut = this.getComponent(UITransform);

```

```

21809         if (!g || !ut) {
21810             return;
21811         }
21812         g.clear();
21813         const w = ut.width;
21814         const h = ut.height;
21815         const left = -ut.anchorX * w;
21816         const bottom = -ut.anchorY * h;
21817         const layout = computeMenuTitleRightBeamLayout(
21818             ut,
21819             left,
21820             bottom,
21821             w,
21822             h,
21823             this._resolveBeamBoundsNode(),
21824         );
21825         drawMenuTitleWhiteSourceRightBeam(g, left, bottom, w, h, layout);
21826         this._syncSpark(layout);
21827     }
21828 }
21829 /** 为 GameTitleIcon2 节点挂上光源图标绘制 */
21830 export function ensureMenuGameTitleIcon2View(iconNode: Node | null): void {
21831     if (!iconNode?.isValid) {
21832         return;
21833     }
21834     if (!iconNode.getComponent(MenuGameTitleIcon2View)) {
21835         iconNode.addComponent(MenuGameTitleIcon2View);
21836     }
21837 }
21838 // ----- assets/Scripts/MenuGameTitleIconView.ts -----
21839 import { _decorator, Component, Graphics, Node, Sprite, UITransform } from 'cc';
21840 import { Direction } from './Games/OpticalPuzzle/Core/OpticalPuzzleTypes';
21841 import { drawPlayerEyes } from './Games/OpticalPuzzle/Presentation/OpticalPuzzlePlayerGlyph';
21842 const { ccclass } = _decorator;
21843 /** 与 OpticalPuzzleBoardView 主角闲置眨眼一致 */
21844 const MENU_PLAYER_BLINK_INTERVAL = 4;
21845 const MENU_PLAYER_BLINK_DURATION = 0.25;
21846 /** Menu/MainPanel/GameTitleIcon: 主角图标（睁眼、朝左看，每 4s 眨一次） */
21847 @ccclass('MenuGameTitleIconView')
21848 export class MenuGameTitleIconView extends Component {
21849     private _graphics: Graphics | null = null;
21850     /** 眨眼周期计时（秒） */
21851     private _blinkClock = 0;
21852     protected onLoad(): void {
21853         this._disableRasterPlaceholder();
21854         this._ensureGraphics();
21855         this._redraw();
21856     }
21857     protected onEnable(): void {
21858         this._blinkClock = 0;
21859         this._redraw();
21860     }
21861     protected update(dt: number): void {
21862         this._blinkClock += dt;
21863         this._redraw();
21864     }
21865     private _currentBlinkAmount(): number {
21866         const cycle = MENU_PLAYER_BLINK_INTERVAL + MENU_PLAYER_BLINK_DURATION;

```

```

21867         const t = this._blinkClock % cycle;
21868         if (t < MENU_PLAYER_BLINK_INTERVAL) {
21869             return 0;
21870         }
21871         const p = (t - MENU_PLAYER_BLINK_INTERVAL) / MENU_PLAYER_BLINK_DURATION;
21872         return Math.sin(Math.min(1, p) * Math.PI);
21873     }
21874     private _disableRasterPlaceholder(): void {
21875         const splash = this.node.getChildByName('SpriteSplash');
21876         if (splash?.isValid) {
21877             splash.active = false;
21878         }
21879         const sprite = this.getComponent(Sprite);
21880         if (sprite) {
21881             sprite.enabled = false;
21882         }
21883     }
21884     private _ensureGraphics(): Graphics | null {
21885         if (!this._graphics?.isValid) {
21886             this._graphics = this.getComponent(Graphics) ?? this.addComponent(Graphics);
21887         }
21888         return this._graphics;
21889     }
21890     private _redraw(): void {
21891         const g = this._ensureGraphics();
21892         const ut = this.getComponent(UITransform);
21893         if (!g || !ut) {
21894             return;
21895         }
21896         g.clear();
21897         const w = ut.width;
21898         const h = ut.height;
21899         const size = Math.min(w, h);
21900         const left = -ut.anchorX * w + (w - size) * 0.5;
21901         const bottom = -ut.anchorY * h + (h - size) * 0.5;
21902         drawPlayerEyes(
21903             g,
21904             left,
21905             bottom,
21906             size,
21907             Direction.Left,
21908             this._currentBlinkAmount(),
21909             false,
21910         );
21911     }
21912 }
21913 /** 为 GameTitleIcon 节点挂上主角图标绘制 */
21914 export function ensureMenuGameTitleIconView(iconNode: Node | null): void {
21915     if (!iconNode?.isValid) {
21916         return;
21917     }
21918     if (!iconNode.getComponent(MenuGameTitleIconView)) {
21919         iconNode.addComponent(MenuGameTitleIconView);
21920     }
21921 }
21922 // ----- assets/Scripts/MenuGameTitleView.ts -----
21923 import { _decorator, Component, Graphics, Node, Sprite, UITransform } from 'cc';
21924 import { drawGameTitleOutline } from './Games/OpticalPuzzle/Presentation/MenuGameTitleGlyph';

```

```

21925 const { ccclass } = _decorator;
21926 /** Menu/MainPanel/GameTitle: 矢量轮廓标题（替代 PNG Sprite） */
21927 @ccclass('MenuGameTitleView')
21928 export class MenuGameTitleView extends Component {
21929     private _graphics: Graphics | null = null;
21930     protected onLoad(): void {
21931         this._disableRasterTitle();
21932         this._ensureGraphics();
21933         this._redraw();
21934     }
21935     protected onEnable(): void {
21936         this._redraw();
21937     }
21938     private _disableRasterTitle(): void {
21939         const splash = this.node.getChildByName('SpriteSplash');
21940         if (splash?.isValid) {
21941             splash.active = false;
21942         }
21943         const sprite = this.getComponent(Sprite);
21944         if (sprite) {
21945             sprite.enabled = false;
21946         }
21947     }
21948     private _ensureGraphics(): Graphics | null {
21949         if (!this._graphics?.isValid) {
21950             this._graphics = this.getComponent(Graphics) ?? this.addComponent(Graphics);
21951         }
21952         return this._graphics;
21953     }
21954     private _redraw(): void {
21955         const g = this._ensureGraphics();
21956         const ut = this.getComponent(UITransform);
21957         if (!g || !ut) {
21958             return;
21959         }
21960         g.clear();
21961         const w = ut.width;
21962         const h = ut.height;
21963         const left = -ut.anchorX * w;
21964         const bottom = -ut.anchorY * h;
21965         drawGameTitleOutline(g, left, bottom, w, h);
21966     }
21967 }
21968 /** 为 GameTitle 节点挂上矢量标题绘制 */
21969 export function ensureMenuGameTitleView(titleNode: Node | null): void {
21970     if (!titleNode?.isValid) {
21971         return;
21972     }
21973     if (!titleNode.getComponent(MenuGameTitleView)) {
21974         titleNode.addComponent(MenuGameTitleView);
21975     }
21976 }
21977 // ----- assets/Scripts/MenuLevelSelectButtonView.ts -----
21978 import {
21979     _decorator,
21980     Button,
21981     Component,
21982     Graphics,

```

```

21983     Node,
21984     UITransform,
21985 } from 'cc';
21986 import { HudButtonPressController } from
21987 './Games/OpticalPuzzle/Presentation/OpticalPuzzleHudButtonPressController';
21988 import { drawMenuLevelSelectButtonGlyph } from
21989 './Games/OpticalPuzzle/Presentation/MenuLevelSelectButtonGlyph';
21990 const { ccclass } = _decorator;
21991 /** 与开始键 Button.zoomScale 一致 */
21992 const LEVEL_SELECT_MENU_BUTTON_ZOOM_SCALE = 0.95;
21993 /** Menu/MainPanel/BtnLevelSelect: 圆形键帽 + 四宫格 icon (点击由 MenuManager 绑定) */
21994 @ccclass('MenuLevelSelectButtonView')
21995 export class MenuLevelSelectButtonView extends Component {
21996     private _graphics: Graphics | null = null;
21997     private _pressed = false;
21998     private _pressCtrl: HudButtonPressController | null = null;
21999     protected onLoad(): void {
22000         this._hidePlaceholderSplash();
22001         this._ensureButton();
22002         this._pressCtrl = new HudButtonPressController(this.node, (pressed) => {
22003             this._pressed = pressed;
22004             this._redraw();
22005         });
22006         this._redraw();
22007     }
22008     protected onEnable(): void {
22009         this._pressCtrl?.bind();
22010         this._redraw();
22011     }
22012     protected onDisable(): void {
22013         this._pressCtrl?.unbind();
22014         this._pressed = false;
22015     }
22016     protected onDestroy(): void {
22017         this._pressCtrl?.unbind();
22018     }
22019     private _hidePlaceholderSplash(): void {
22020         const splash = this.node.getChildByName('SpriteSplash') ?? this.node.getChildByName('bg');
22021         if (splash?.isValid) {
22022             splash.active = false;
22023         }
22024     }
22025     private _ensureButton(): void {
22026         let btn = this.getComponent(Button);
22027         if (!btn) {
22028             btn = this.addComponent(Button);
22029             btn.transition = Button.Transition.SCALE;
22030         }
22031         btn.zoomScale = LEVEL_SELECT_MENU_BUTTON_ZOOM_SCALE;
22032         btn.transition = Button.Transition.SCALE;
22033     }
22034     private _ensureGraphics(): Graphics | null {
22035         if (!this._graphics?.isValid) {
22036             this._graphics = this.getComponent(Graphics) ?? this.addComponent(Graphics);
22037         }
22038         return this._graphics;
22039     }
22040     private _redraw(): void {

```

```

22041         const g = this._ensureGraphics();
22042         const ut = this.getComponent(UITransform);
22043         if (!g || !ut) {
22044             return;
22045         }
22046         g.clear();
22047         const w = ut.width;
22048         const h = ut.height;
22049         const left = -ut.anchorX * w;
22050         const bottom = -ut.anchorY * h;
22051         drawMenuLevelSelectButtonGlyph(g, left, bottom, w, h, this._pressed);
22052     }
22053 }
22054 /** 为 MainPanel/BtnLevelSelect 挂上键帽绘制与按压缩放 */
22055 export function ensureMenuLevelSelectButtonView(btnLevelSelect: Node | null): void {
22056     if (!btnLevelSelect?.isValid) {
22057         return;
22058     }
22059     if (!btnLevelSelect.getComponent(MenuLevelSelectButtonView)) {
22060         btnLevelSelect.addComponent(MenuLevelSelectButtonView);
22061     }
22062 }
22063 // ----- assets/Scripts/MenuOverlayWindow.ts -----
22064 import { _decorator, Component, Node } from 'cc';
22065 import { RoundBox } from '../RoundBox';
22066 import { EVENT_ENUM, AUDIO_EFFECT_ENUM } from './Utils/Enum';
22067 import { PLAY_AUDIO } from './Utils/Event';
22068 const { ccclass, property } = _decorator;
22069 /**
22070  * 菜单简单弹层：挂在窗口根节点上，关闭键（及可选子节点 `bg` 衬底）点击后隐藏本节点
22071  */
22072 @ccclass('MenuOverlayWindow')
22073 export class MenuOverlayWindow extends Component {
22074     /** 关闭按钮 */
22075     @property(Node)
22076     closeButtonNode: Node = null;
22077     private _backdropBgNode: Node | null = null;
22078     protected onLoad(): void {
22079         this._backdropBgNode = this.node.getChildByName('bg') ?? null;
22080         if (this._backdropBgNode) {
22081             this._backdropBgNode.on(Node.EventType.TOUCH_END, this.onCloseClick, this);
22082         }
22083         if (this.closeButtonNode) {
22084             this.closeButtonNode.on(Node.EventType.TOUCH_END, this.onCloseClick, this);
22085         }
22086     }
22087     protected onEnable(): void {
22088         // 根节点从场景初始 inactive 首次打开时，子节点 RoundBox 可能首帧未拿到 texture/chunk
22089         this.scheduleOnce(this.refreshChildRoundBoxes, 0);
22090     }
22091     private refreshChildRoundBoxes(): void {
22092         if (!this.node?.isValid || !this.node.activeInHierarchy) {
22093             return;
22094         }
22095         const list = this.node.getComponentsInChildren(RoundBox);
22096         for (let i = 0; i < list.length; i++) {
22097             list[i].forceRefreshRender();
22098         }

```

```

22099     }
22100     protected onDestroy(): void {
22101         if (this._backdropBgNode?.isValid) {
22102             this._backdropBgNode.off(Node.EventType.TOUCH_END, this.onCloseClick, this);
22103         }
22104         this._backdropBgNode = null;
22105         if (this.closeButtonNode?.isValid) {
22106             this.closeButtonNode.off(Node.EventType.TOUCH_END, this.onCloseClick, this);
22107         }
22108     }
22109     private onCloseClick(): void {
22110         PLAY_AUDIO.emit(EVENT_ENUM.PLAY_AUDIO, AUDIO_EFFECT_ENUM.CLICK_BUTTON);
22111         this.node.active = false;
22112     }
22113 }
22114 // ----- assets/Scripts/MenuShareButtonView.ts -----
22115 import {
22116     _decorator,
22117     Button,
22118     Component,
22119     Graphics,
22120     Node,
22121     UITransform,
22122 } from 'cc';
22123 import { HudButtonPressController } from
22124 './Games/OpticalPuzzle/Presentation/OpticalPuzzleHudButtonPressController';
22125 import { drawMenuShareButtonGlyph } from './Games/OpticalPuzzle/Presentation/MenuShareButtonGlyph';
22126 const { ccclass } = _decorator;
22127 /** 与开始键 Button.zoomScale 一致 */
22128 const SHARE_MENU_BUTTON_ZOOM_SCALE = 0.95;
22129 /** Menu/MainPanel/BtnShare: 圆形键帽 + 分享 icon (点击由 MenuManager 绑定) */
22130 @ccclass('MenuShareButtonView')
22131 export class MenuShareButtonView extends Component {
22132     private _graphics: Graphics | null = null;
22133     private _pressed = false;
22134     private _pressCtrl: HudButtonPressController | null = null;
22135     protected onLoad(): void {
22136         this._hidePlaceholderSplash();
22137         this._ensureButton();
22138         this._pressCtrl = new HudButtonPressController(this.node, (pressed) => {
22139             this._pressed = pressed;
22140             this._redraw();
22141         });
22142         this._redraw();
22143     }
22144     protected onEnable(): void {
22145         this._pressCtrl?.bind();
22146         this._redraw();
22147     }
22148     protected onDisable(): void {
22149         this._pressCtrl?.unbind();
22150         this._pressed = false;
22151     }
22152     protected onDestroy(): void {
22153         this._pressCtrl?.unbind();
22154     }
22155     private _hidePlaceholderSplash(): void {
22156         const splash = this.node.getChildByName('SpriteSplash') ?? this.node.getChildByName('bg');

```

```

22157         if (splash?.isValid) {
22158             splash.active = false;
22159         }
22160     }
22161     private _ensureButton(): void {
22162         let btn = this.getComponent(Button);
22163         if (!btn) {
22164             btn = this.addComponent(Button);
22165             btn.transition = Button.Transition.SCALE;
22166         }
22167         btn.zoomScale = SHARE_MENU_BUTTON_ZOOM_SCALE;
22168         btn.transition = Button.Transition.SCALE;
22169     }
22170     private _ensureGraphics(): Graphics | null {
22171         if (!this._graphics?.isValid) {
22172             this._graphics = this.getComponent(Graphics) ?? this.addComponent(Graphics);
22173         }
22174         return this._graphics;
22175     }
22176     private _redraw(): void {
22177         const g = this._ensureGraphics();
22178         const ut = this.getComponent(UITransform);
22179         if (!g || !ut) {
22180             return;
22181         }
22182         g.clear();
22183         const w = ut.width;
22184         const h = ut.height;
22185         const left = -ut.anchorX * w;
22186         const bottom = -ut.anchorY * h;
22187         drawMenuShareButtonGlyph(g, left, bottom, w, h, this._pressed);
22188     }
22189 }
22190 /** 为 MainPanel/BtnShare 挂上键帽绘制与按压缩放 */
22191 export function ensureMenuShareButtonView(btnShare: Node | null): void {
22192     if (!btnShare?.isValid) {
22193         return;
22194     }
22195     if (!btnShare.getComponent(MenuShareButtonView)) {
22196         btnShare.addComponent(MenuShareButtonView);
22197     }
22198 }
22199 // ----- assets/Scripts/MenuStartButtonView.ts -----
22200 import {
22201     _decorator,
22202     Button,
22203     Component,
22204     Graphics,
22205     Node,
22206     UITransform,
22207 } from 'cc';
22208 import { HudButtonPressController } from
22209 './Games/OpticalPuzzle/Presentation/OpticalPuzzleHudButtonPressController';
22210 import { drawMenuStartButtonGlyph } from './Games/OpticalPuzzle/Presentation/MenuStartButtonGlyph';
22211 const { ccclass } = _decorator;
22212 /** 与 nextlevel / 四向键 Button.zoomScale 一致 */
22213 const START_BUTTON_ZOOM_SCALE = 0.95;
22214 /** Menu/MainPanel/BtnStart: 圆形键帽（点击逻辑由 MenuManager 绑定） */

```

```

22215 @cclass('MenuStartButtonView')
22216 export class MenuStartButtonView extends Component {
22217     private _graphics: Graphics | null = null;
22218     private _pressed = false;
22219     private _pressCtrl: HudButtonPressController | null = null;
22220     protected onLoad(): void {
22221         this._hidePlaceholderBg();
22222         this._ensureButton();
22223         this._pressCtrl = new HudButtonPressController(this.node, (pressed) => {
22224             this._pressed = pressed;
22225             this._redraw();
22226         });
22227         this._redraw();
22228     }
22229     protected onEnable(): void {
22230         this._pressCtrl?.bind();
22231         this._redraw();
22232     }
22233     protected onDisable(): void {
22234         this._pressCtrl?.unbind();
22235         this._pressed = false;
22236     }
22237     protected onDestroy(): void {
22238         this._pressCtrl?.unbind();
22239     }
22240     private _hidePlaceholderBg(): void {
22241         const bg = this.node.getChildByName('bg');
22242         if (bg?.isValid) {
22243             bg.active = false;
22244         }
22245     }
22246     private _ensureButton(): void {
22247         let btn = this.getComponent(Button);
22248         if (!btn) {
22249             btn = this.addComponent(Button);
22250             btn.transition = Button.Transition.SCALE;
22251         }
22252         btn.zoomScale = START_BUTTON_ZOOM_SCALE;
22253         btn.transition = Button.Transition.SCALE;
22254     }
22255     private _ensureGraphics(): Graphics | null {
22256         if (!this._graphics?.isValid) {
22257             this._graphics = this.getComponent(Graphics) ?? this.addComponent(Graphics);
22258         }
22259         return this._graphics;
22260     }
22261     private _redraw(): void {
22262         const g = this._ensureGraphics();
22263         const ut = this.getComponent(UITransform);
22264         if (!g || !ut) {
22265             return;
22266         }
22267         g.clear();
22268         const w = ut.width;
22269         const h = ut.height;
22270         const left = -ut.anchorX * w;
22271         const bottom = -ut.anchorY * h;
22272         drawMenuStartButtonGlyph(g, left, bottom, w, h, this._pressed);

```

```

22273     }
22274 }
22275 /** 为 MainPanel/BtnStart 挂上键帽绘制与按压缩放 */
22276 export function ensureMenuStartButtonView(btnStart: Node | null): void {
22277     if (!btnStart?.isValid) {
22278         return;
22279     }
22280     if (!btnStart.getComponent(MenuStartButtonView)) {
22281         btnStart.addComponent(MenuStartButtonView);
22282     }
22283 }
22284 // ----- assets/Scripts/Utils/Enum.ts -----
22285 /**
22286  * 全局枚举集中定义（与 project-rules 一致）。
22287  * 光学解谜专用事件名使用 OPTICAL_ 前缀，避免与通用事件混淆。
22288  */
22289 export enum EVENT_ENUM {
22290     /** 音效播放（载荷为 AUDIO_EFFECT_ENUM） */
22291     PLAY_AUDIO = 'PLAY_AUDIO',
22292     /** 背景音乐切换 */
22293     PLAY_BGM = 'PLAY_BGM',
22294     /** 全局游戏状态变更 */
22295     GAME_STATE_CHANGE = 'GAME_STATE_CHANGE',
22296     /** 光学解谜棋盘快照更新（载荷为 OpticalBoardSnapshot，由 Presentation 监听） */
22297     OPTICAL_SNAPSHOT_CHANGED = 'OPTICAL_SNAPSHOT_CHANGED',
22298     /** Toast 提示（载荷 { message, bgWidth?, localY? }） */
22299     SHOW_TOAST = 'SHOW_TOAST',
22300 }
22301 /** 音效类型（按需扩展；播放经 PLAY_AUDIO 事件，由 MusicManager 映射 AudioClip） */
22302 export enum AUDIO_EFFECT_ENUM {
22303     /** UI 通用按钮点击 */
22304     CLICK_BUTTON = 'CLICK_BUTTON',
22305     /** 四向键：移动成功（走一格或推动元件） */
22306     OPTICAL_MOVE_SUCCESS = 'OPTICAL_MOVE_SUCCESS',
22307     /** 四向键：移动失败（与主角「><」阻拦脸同一时机） */
22308     OPTICAL_MOVE_FAIL = 'OPTICAL_MOVE_FAIL',
22309     /** 单个目标灯被点亮 */
22310     OPTICAL_TARGET_LIT = 'OPTICAL_TARGET_LIT',
22311     /** 关卡通关成功 */
22312     OPTICAL_LEVEL_COMPLETE = 'OPTICAL_LEVEL_COMPLETE',
22313 }
22314 /** 背景音乐种类（PLAY_BGM 刷新用；全局单曲 Bg.mp3，不再区分 Menu/Game） */
22315 export enum BGM_KIND_ENUM {
22316     GLOBAL = 'GLOBAL',
22317 }
22318 /** 场景名与 build 设置中场景文件名一致 */
22319 export enum SCENE_ENUM {
22320     MENU = 'Menu',
22321     /** 局内场景占位，创建 Game 场景后改为实际名称 */
22322     GAME = 'Game',
22323 }
22324 /** 与 project-rules 示例一致的全局游戏状态（跨玩法可复用） */
22325 export enum GAME_STATE_ENUM {
22326     INIT,
22327     RUNNING,
22328     PAUSED,
22329     LOSED,
22330     COMPLETE,

```

```

22331 }
22332 // ----- assets/Scripts/Utils/Event.ts -----
22333 import { EventTarget } from 'cc';
22334 /** 全局音效 */
22335 export const PLAY_AUDIO = new EventTarget();
22336 /** 背景音乐 */
22337 export const PLAY_BGM = new EventTarget();
22338 /** 全局状态 */
22339 export const GAME_STATE_CHANGE = new EventTarget();
22340 /** 光学解谜: 棋盘 / HUD 刷新等 */
22341 export const OPTICAL_PUZZLE = new EventTarget();
22342 /** 全局 Toast 提示 (载荷 { message, bgWidth?, localY?, 由局内 Presentation 转发到 ToastManager) */
22343 export const SHOW_TOAST = new EventTarget();
22344 // ----- assets/Scripts/Utils/Utils.ts -----
22345 /**
22346  * 通用工具函数 (与 project-rules 目录约定一致)。
22347  */
22348 export function clamp(value: number, min: number, max: number): number {
22349     return Math.max(min, Math.min(max, value));
22350 }
22351 // ----- assets/Scripts/Utils/WeChatMiniGameAds.ts -----
22352 import { director } from 'cc';
22353 import { EVENT_ENUM, SCENE_ENUM } from './Enum';
22354 import { PLAY_BGM } from './Event';
22355 /**
22356  * 微信小游戏流量主 — 激励视频 (参考 MemoMaster)。
22357  * 文档: https://developers.weixin.qq.com/minigame/dev/api/ad/wx.createRewardedVideoAd.html
22358  */
22359 const LOG = '[WeChatMiniGameAds]';
22360 /**
22361  * 开发测试: true 时不拉真实广告, 直接 resolve true。
22362  * 发版前务必改回 false。
22363  */
22364 export let USE_DEBUG MOCK_REWARDED_AD_SUCCESS = true; // mock 广告返回 true
22365 /** 激励式视频广告位 id (流量主后台; 留空则 show 直接 resolve false) */
22366 export let WECHAT_REWARDED_VIDEO_AD_UNIT_ID = '';
22367 interface RewardedVideoCloseResult {
22368     isEnded?: boolean;
22369 }
22370 interface IRewardedVideoAd {
22371     load(): Promise<void>;
22372     show(): Promise<void>;
22373     onClose(listener: (res?: RewardedVideoCloseResult) => void): void;
22374     offClose(listener: (res?: RewardedVideoCloseResult) => void): void;
22375     onError(listener: (err: unknown) => void): void;
22376 }
22377 interface IWxAds {
22378     createRewardedVideoAd?(opt: { adUnitId: string; multiton?: boolean }): IRewardedVideoAd | null;
22379 }
22380 function getWx(): IWxAds | undefined {
22381     const g = globalThis as unknown as { wx?: IWxAds };
22382     return g.wx;
22383 }
22384 let _rewarded: IRewardedVideoAd | null = null;
22385 let _rewardedErrorBound = false;
22386 let _rewardedShowInFlight = false;
22387 let _warnedEmptyRewardedId = false;
22388 let _firstPreloadTimer: ReturnType<typeof setTimeout> | null = null;

```

```
22389 const REWARDED_PRELOAD_DELAY_MS = 2000;
22390 const REWARDED_RESUME_BGM_DELAY_MS = 100;
22391 function scheduleResumeBgmAfterRewardedAd(): void {
22392     setTimeout(() => {
22393         const scene = director.getScene()?.name;
22394         if (scene === SCENE_ENUM.MENU || scene === SCENE_ENUM.GAME) {
22395             PLAY_BGM.emit(EVENT_ENUM.PLAY_BGM, scene);
22396         }
22397     }, REWARDED_RESUME_BGM_DELAY_MS);
22398 }
22399 function bindRewardedErrorOnce(ad: IRewardedVideoAd): void {
22400     if (_rewardedErrorBound) {
22401         return;
22402     }
22403     _rewardedErrorBound = true;
22404     ad.onError((err: unknown) => {
22405         console.warn(LOG, 'Rewarded onError', err);
22406     });
22407 }
22408 function getOrCreateRewarded(adUnitId: string): IRewardedVideoAd | null {
22409     const wxApi = getWx();
22410     if (typeof wxApi?.createRewardedVideoAd !== 'function') {
22411         return null;
22412     }
22413     if (!_rewarded) {
22414         const ad = wxApi.createRewardedVideoAd({ adUnitId });
22415         if (!ad) {
22416             return null;
22417         }
22418         _rewarded = ad;
22419         bindRewardedErrorOnce(ad);
22420     }
22421     return _rewarded;
22422 }
22423 export function isWeChatRewardedVideoAdConfigured(): boolean {
22424     return WECHAT_REWARDED_VIDEO_AD_UNIT_ID.trim().length > 0;
22425 }
22426 export function cancelWeChatRewardedVideoPreloadSchedule(): void {
22427     if (_firstPreloadTimer !== null) {
22428         clearTimeout(_firstPreloadTimer);
22429         _firstPreloadTimer = null;
22430     }
22431 }
22432 export function initWeChatRewardedVideoAd(delayMs: number): void {
22433     cancelWeChatRewardedVideoPreloadSchedule();
22434     const run = (): void => {
22435         _firstPreloadTimer = null;
22436         preloadWeChatRewardedVideo();
22437     };
22438     if (delayMs <= 0) {
22439         run();
22440         return;
22441     }
22442     _firstPreloadTimer = setTimeout(run, delayMs);
22443 }
22444 /** Game 场景：延迟预载，避开开局动画 */
22445 export function scheduleWeChatRewardedVideoPreloadForGame(): void {
22446     initWeChatRewardedVideoAd(REWARDED_PRELOAD_DELAY_MS);
```

```
22447 }
22448 /** 展示激励视频；完整观看返回 true，否则 false */
22449 export function showWeChatRewardedVideo(): Promise<boolean> {
22450   if (USE_DEBUG MOCK_REWARDED_AD_SUCCESS) {
22451     console.warn(LOG, 'USE_DEBUG MOCK_REWARDED_AD_SUCCESS=true，模拟观看成功');
22452     return Promise.resolve(true);
22453   }
22454   const id = WECHAT_REWARDED_VIDEO_AD_UNIT_ID.trim();
22455   if (!id) {
22456     if (!_warnedEmptyRewardedId) {
22457       _warnedEmptyRewardedId = true;
22458       console.warn(LOG, 'WECHAT_REWARDED_VIDEO_AD_UNIT_ID 为空，已跳过激励视频');
22459     }
22460     return Promise.resolve(false);
22461   }
22462   const wxApi = getWx();
22463   if (typeof wxApi?.createRewardedVideoAd !== 'function') {
22464     return Promise.resolve(false);
22465   }
22466   if (_rewardedShowInFlight) {
22467     console.warn(LOG, '激励视频已在展示/加载中，忽略本次');
22468     return Promise.resolve(false);
22469   }
22470   const ad = getOrCreateRewarded(id);
22471   if (!ad) {
22472     return Promise.resolve(false);
22473   }
22474   _rewardedShowInFlight = true;
22475   return new Promise<boolean>((resolve) => {
22476     let settled = false;
22477     const finish = (ok: boolean): void => {
22478       if (settled) {
22479         return;
22480       }
22481       settled = true;
22482       _rewardedShowInFlight = false;
22483       ad.offClose(onClose);
22484       resolve(ok);
22485     };
22486     const onClose = (res?: RewardedVideoCloseResult): void => {
22487       const ok = !res || res.isEnded === true;
22488       finish(ok);
22489       scheduleResumeBgmAfterRewardedAd();
22490       preloadWeChatRewardedVideo();
22491     };
22492     ad.onClose(onClose);
22493     ad.show()
22494       .catch(() => ad.load().then(() => ad.show()))
22495       .catch((err: unknown) => {
22496         console.warn(LOG, '激励视频 广告显示失败', err);
22497         finish(false);
22498       });
22499   });
22500 }
22501 export function preloadWeChatRewardedVideo(): void {
22502   const id = WECHAT_REWARDED_VIDEO_AD_UNIT_ID.trim();
22503   if (!id) {
22504     return;
```

```

22505     }
22506     const ad = getOrCreateRewarded(id);
22507     if (!ad) {
22508         return;
22509     }
22510     ad.load().catch((err: unknown) => {
22511         console.warn(LOG, 'Rewarded preload 失败', err);
22512     });
22513 }
22514 // ----- assets/Scripts/Utils/WeChatRewardedVideoAd.ts -----
22515 /**
22516  * 兼容入口：激励视频实现见 `WeChatMiniGameAds.ts`（与 MemoMaster 一致）。
22517  */
22518 export {
22519     WECHAT_REWARDED_VIDEO_AD_UNIT_ID,
22520     USE_DEBUG MOCK_REWARDED_AD_SUCCESS,
22521     isWeChatRewardedVideoAdConfigured,
22522     initWeChatRewardedVideoAd,
22523     cancelWeChatRewardedVideoPreloadSchedule,
22524     scheduleWeChatRewardedVideoPreloadForGame,
22525     showWeChatRewardedVideo,
22526     preloadWeChatRewardedVideo,
22527 } from './WeChatMiniGameAds';
22528 // ----- assets/Scripts/Utils/WeChatShare.ts -----
22529 /**
22530  * 微信小游戏：右上角「转发 / 朋友圈 / 复制链接」需 showShareMenu + 分享回调，否则常为灰色。
22531  * `ensureWxShareMenuAndPassiveHooks` 可多次调用：重复执行 `showShareMenu`（点击前再补一次可避免「不调起面板」）。
22532  */
22533 /*
22534 与转发、朋友圈展示文案一致时可改此处 */
22535 const SHARE_TITLE_FRIEND = '光学解谜 — 你能点亮全部接收器吗？';
22536 const SHARE_TITLE_TIMELINE = '光学解谜 — 推光路大挑战';
22537 /**
22538  * 分享预览图（imageUrl）。
22539  * 留空：不传 imageUrl，微信会用当前游戏画面截图（稳定，推荐默认）
22540  */
22541 const SHARE_PREVIEW_IMAGE_HTTPS = '';
22542 const SHARE_FOREGROUND_DEBOUNCE_MS = 320;
22543 const SHARE_RETURN_AFTER_HIDE_MS = 80;
22544 const SHARE_NO_HIDE_FALLBACK_MS = 520;
22545 const SHARE_FAIL_TIMEOUT_MS = 25000;
22546 interface ShareAppMessageOptions {
22547     title?: string;
22548     imageUrl?: string;
22549     query?: string;
22550     imageUrlId?: string;
22551 }
22552 interface IWxMiniGameShare {
22553     showShareMenu(opt: { withShareTicket?: boolean; menus?: string[] }): void;
22554     onShareAppMessage(fn: () => { title?: string; imageUrl?: string; query?: string }): void;
22555     onShareTimeline?(fn: () => { title?: string; imageUrl?: string; query?: string }): void;
22556     shareAppMessage?(opt: ShareAppMessageOptions): void;
22557     onShow?(fn: (res?: unknown) => void): void;
22558     onHide?(fn: () => void): void;
22559     offShow?(fn: (res?: unknown) => void): void;
22560     offHide?(fn: () => void): void;
22561 }
22562 function getWx(): IWxMiniGameShare | undefined {

```

```
22563     const g = globalThis as unknown as { wx?: IWxMiniGameShare };
22564     return g.wx;
22565 }
22566 function buildFriendSharePayload(): { title: string; imageUrl?: string } {
22567     const o: { title: string; imageUrl?: string } = { title: SHARE_TITLE_FRIEND };
22568     if (SHARE_PREVIEW_IMAGE_HTTPS) {
22569         o.imageUrl = SHARE_PREVIEW_IMAGE_HTTPS;
22570     }
22571     return o;
22572 }
22573 let _hooksBound = false;
22574 function ensureWxShareMenuAndPassiveHooks(): void {
22575     const wxApi = getWx();
22576     if (!wxApi) {
22577         return;
22578     }
22579     try {
22580         if (typeof wxApi.showShareMenu === 'function') {
22581             wxApi.showShareMenu({
22582                 withShareTicket: true,
22583                 menus: ['shareAppMessage', 'shareTimeline'],
22584             });
22585         }
22586     } catch {
22587         /* 部分环境首帧不可用，稍后点击再调 */
22588     }
22589     if (_hooksBound) {
22590         return;
22591     }
22592     if (typeof wxApi.onShareAppMessage !== 'function') {
22593         return;
22594     }
22595     wxApi.onShareAppMessage(() => buildFriendSharePayload());
22596     const sharePayloadTimeline = () => {
22597         const o: { title: string; imageUrl?: string } = { title: SHARE_TITLE_TIMELINE };
22598         if (SHARE_PREVIEW_IMAGE_HTTPS) {
22599             o.imageUrl = SHARE_PREVIEW_IMAGE_HTTPS;
22600         }
22601         return o;
22602     };
22603     if (typeof wxApi.onShareTimeline === 'function') {
22604         wxApi.onShareTimeline(sharePayloadTimeline);
22605     }
22606     _hooksBound = true;
22607 }
22608 export function trySettlePendingShareAfterWxOnShow(): void {
22609     handleShareAwaitOnForeground();
22610 }
22611 type ShareAwait = {
22612     onSuccess: () => void;
22613     onFail: (reason?: string) => void;
22614     startAt: number;
22615     hideAt: number | null;
22616     settled: boolean;
22617     hideListener: (() => void) | null;
22618     failTimer: ReturnType<typeof setTimeout> | null;
22619 };
22620 let _shareAwait: ShareAwait | null = null;
```

```
22621 function cancelShareAwaitHooks(s: ShareAwait): void {
22622     const wxApi = getWx();
22623     if (s.hideListener && typeof wxApi?.offHide === 'function') {
22624         wxApi.offHide(s.hideListener);
22625     }
22626     s.hideListener = null;
22627     if (s.failTimer !== null) {
22628         clearTimeout(s.failTimer);
22629         s.failTimer = null;
22630     }
22631 }
22632 function cancelPendingShareAwaitSilent(): void {
22633     const s = _shareAwait;
22634     if (!s || s.settled) {
22635         return;
22636     }
22637     s.settled = true;
22638     cancelShareAwaitHooks(s);
22639     _shareAwait = null;
22640 }
22641 function settleShareAwait(mode: 'success' | 'fail', reason?: string): void {
22642     const s = _shareAwait;
22643     if (!s || s.settled) {
22644         return;
22645     }
22646     s.settled = true;
22647     cancelShareAwaitHooks(s);
22648     _shareAwait = null;
22649     if (mode === 'success') {
22650         s.onSuccess();
22651     } else {
22652         s.onFail(reason);
22653     }
22654 }
22655 function handleShareAwaitOnForeground(): void {
22656     const s = _shareAwait;
22657     if (!s || s.settled) {
22658         return;
22659     }
22660     const now = Date.now();
22661     const sinceStart = now - s.startAt;
22662     if (sinceStart < SHARE_FOREGROUND_DEBOUNCE_MS) {
22663         return;
22664     }
22665     if (s.hideAt !== null && now - s.hideAt >= SHARE_RETURN_AFTER_HIDE_MS) {
22666         settleShareAwait('success');
22667         return;
22668     }
22669     if (s.hideAt === null && sinceStart >= SHARE_NO_HIDE_FALLBACK_MS) {
22670         settleShareAwait('success');
22671     }
22672 }
22673 export function initWeChatMiniGameShare(): void {
22674     ensureWxShareMenuAndPassiveHooks();
22675 }
22676 export function invokeWeChatFriendShare(cb: {
22677     onSuccess?: () => void;
22678     onFail?: (reason?: string) => void;
```

```
22679   }): void {
22680       const wxApi = getWx();
22681       if (!wxApi || typeof wxApi.shareAppMessage !== 'function') {
22682           cb.onFail?.('unsupported');
22683           return;
22684       }
22685       ensureWxShareMenuAndPassiveHooks();
22686       cancelPendingShareAwaitSilent();
22687       const onSuccess = cb.onSuccess ?? (() => undefined);
22688       const onFail = cb.onFail ?? (() => undefined);
22689       const hideListener = (): void => {
22690           const s = _shareAwait;
22691           if (!s || s.settled) {
22692               return;
22693           }
22694           s.hideAt = Date.now();
22695       };
22696       const awaitState: ShareAwait = {
22697           onSuccess,
22698           onFail,
22699           startAt: Date.now(),
22700           hideAt: null,
22701           settled: false,
22702           hideListener: null,
22703           failTimer: null,
22704       };
22705       _shareAwait = awaitState;
22706       if (typeof wxApi.onHide === 'function') {
22707           awaitState.hideListener = hideListener;
22708           wxApi.onHide(hideListener);
22709       }
22710       awaitState.failTimer = setTimeout(() => {
22711           settleShareAwait('fail', 'timeout');
22712       }, SHARE_FAIL_TIMEOUT_MS);
22713       try {
22714           const base = buildFriendSharePayload();
22715           wxApi.shareAppMessage({
22716               title: base.title,
22717               ...(base.imageUrl ? { imageUrl: base.imageUrl } : {}),
22718           });
22719       } catch {
22720           settleShareAwait('fail', 'error');
22721       }
22722   }
```